



Computer Systems & Professional Practice

Data Representation



Dr Todd Waugh Ambridge



Module Outline

Computer Systems

1. Data Representation
2. Digital Logic Design
3. CPUs & Program Execution
4. Memory & Storage Systems
5. Processor Architecture of Modern Systems

Professional Practice

- A. Introduction to University Life & Student Support
- B. Academic Skills, Attributes & Integrity
- C. CVs & Career Planning
- D. Professional Technologies
- E. Teamwork, Communication & Presentation
- F. Legal, Social & Ethical Considerations



Module Outline

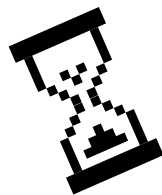
Computer Systems

1. Data Representation
2. Digital Logic Design
3. CPUs & Program Execution
4. Memory & Storage Systems
5. Processor Architecture of Modern Systems



Learning Objective

To explain common
data representations and their
role in computer systems.





Topic Outline



- 01 Computers Speak Binary
- 02 Binary Number System
- 03 Representing Colour & Text
- 04 Binary Arithmetic
- 05 Representing Negative Numbers
- 06 Representing Real Numbers



01001000 01000101
01001100 01001100
01001111 00100001

01

Computers Speak Binary

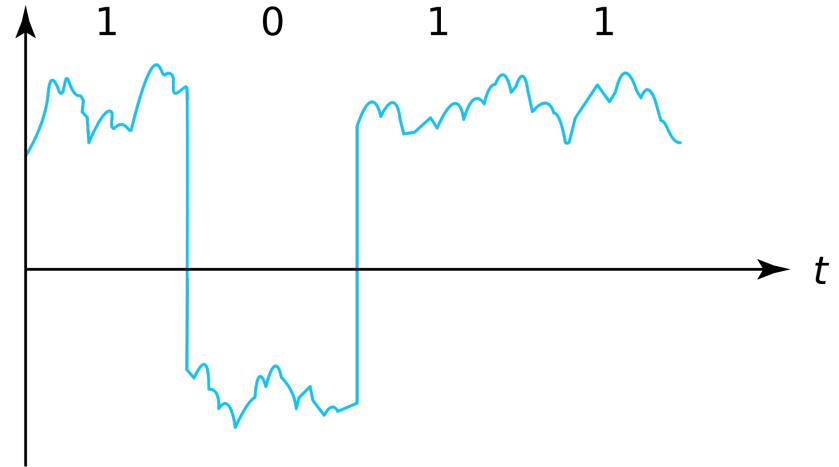
Computers use 0s and 1s to represent
numbers, text, colours, images,
videos and **everything else.**



Computer Hardware is Logical

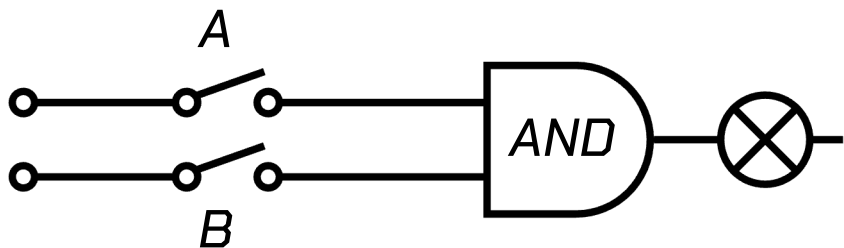
All computer hardware communicates using electronic signals – more specifically, they use **logical signals**.

A logical signal can only be *on* or *off*.





Computer Hardware is Logical



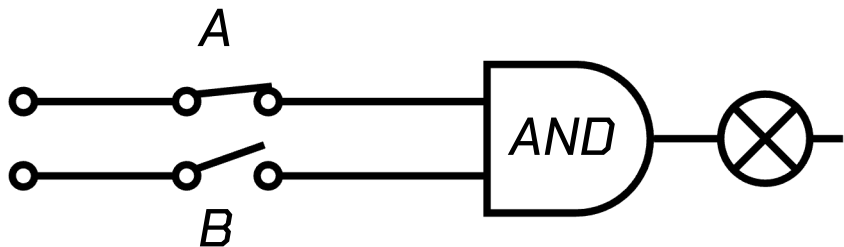
Let's first think about something more basic than a computer: a digital circuit.

This circuit has two switches, *A* and *B* connected to an AND logic gate, which is then connected to a lightbulb *X*.



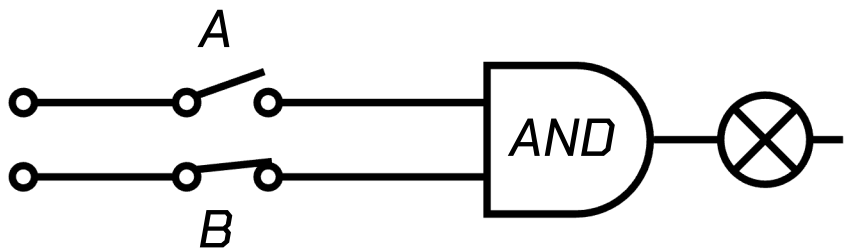
Computer Hardware is Logical

If only *A* is switched *on*, the
lightbulb stays *off*.





Computer Hardware is Logical

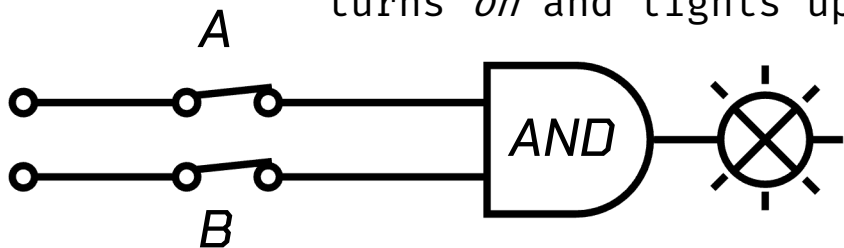


The same happens if only *B* is switched *on*: the lightbulb remains *off*.



Computer Hardware is Logical

But if both *A* and *B* are switched *on*, the lightbulb turns *on* and lights up.



We can model this operation using a *truth table*.

A	B	X
0	0	0
1	0	0
0	1	0
1	1	1

0 represents *off* and 1 represents *on*.



Computer Hardware is Logical

So, digital circuits operate using a logic consisting of only 0s and 1s – a *binary logic*.

The core components of a computer – such as the **CPU** and memory – are made up of millions of digital circuits.

The CPU is where data is *operated on*, this means that data can only be *operated on* using binary (just 0s and 1s).





Computer Hardware is Logical

So, digital circuits operate using a logic consisting of only 0s and 1s – a *binary logic*.

The core components of a computer – such as the CPU and **memory** – are made up of millions of digital circuits.

Memory is where data is *stored*, this means that data can only be *stored* as binary (just 0s and 1s).



Computers Speak Binary

83



S



01010011



In order to *store* and *operate on* everything as *binary* (0s and 1s), we need to be able to **represent** everything as binary.

This means that lots of different items of data (numbers, text, colours, images, videos, etc.) will have a common structure (strings of binary digits).

Computers Speak Binary

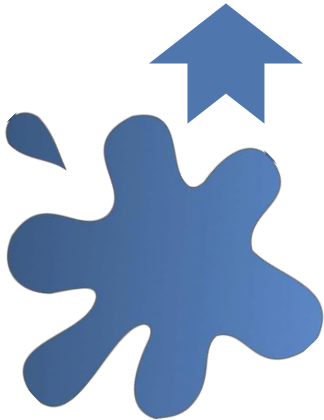
83



01010011



S



If everything is binary, then there will be different representations for different types of data.

A data representation is a method of representing some real world data (text, images, etc.) as binary strings.

Computers Speak Binary

0 or 1
0010 1100
1001

A binary string is a string of any length that consists only of bits.

A 1-bit string is called a **bit**.

An 8-bit string is called a **byte**.

(Less common, but very cute: a 4-bit string is called a **nibble**.)

Computers Speak Binary

0000 0000

...

1111 1111

How many possible byte strings are there?

Possible 1-bit strings = 2

- 0
- 1

Possible 2-bit strings = 4

- 00
- 01
- 10
- 11

Computers Speak Binary

0000 0000

...

1111 1111

How many possible byte strings are there?

Possible 3-bit strings = 8

- 000, 001, 010, 011
- 100, 101, 110, 111

Possible 4-bit strings = 16

- 0000, 0001, 0010, 0011
- 0100, 0101, 0110, 0111
- 1000, 1001, 1010, 1011
- 1100, 1101, 1110, 1111

Computers Speak Binary

0000 0000

...

1111 1111

How many possible byte strings are there?

Possible **1**-bit strings = 2 = 2^1

Possible **2**-bit strings = 4 = 2^2

Possible **3**-bit strings = 8 = 2^3

Possible **4**-bit strings = 16 = 2^4

Possible **byte** strings = $2^8 = 256$

Computers Speak Binary

0000 0000

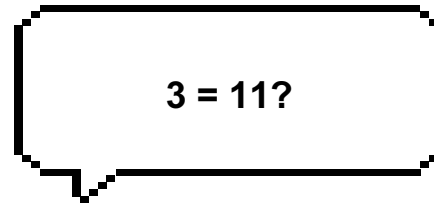
...

1111 1111

Possible byte strings = $2^8 = 256$

So, if we are limited to a single byte to represent something, we can represent 256 different things.

Have you heard of 256 colour images?



02

Binary Number System

Humans use the symbols 0-9 to represent numbers. Computers use the symbols 0-1 to represent numbers.

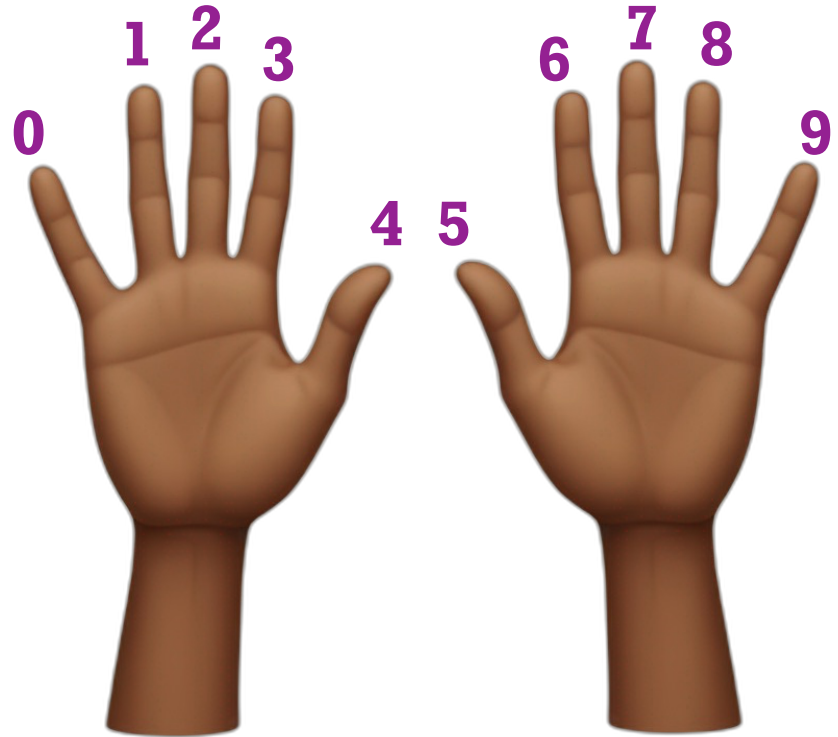
Denary Number System

Our number system is called *denary*.

The prefix 'den' derives from a Latin word meaning "ten options".

Humans use denary because we have ten fingers.

The ten options are called digits.



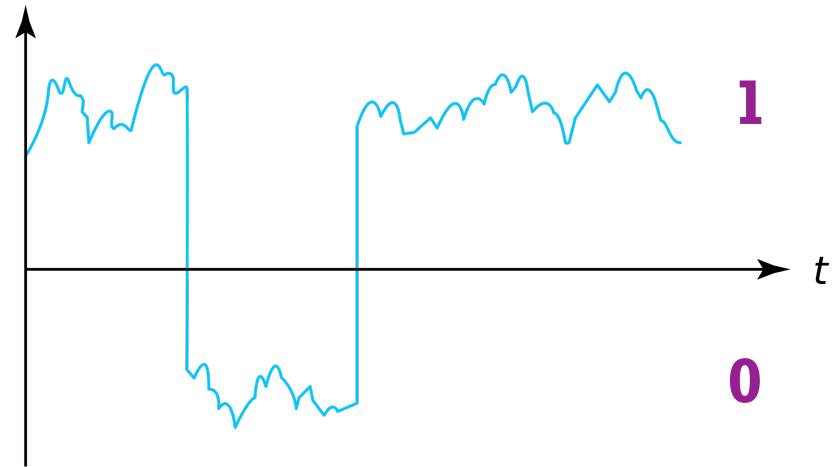
Binary Number System

A computer's number system is called *binary*.

The prefix 'bin' derives from a Latin word meaning "two options".

Computers use binary because electrical signals have two states: 'on' or 'off'.

The two options are called bits.





Binary Number System

In every number system, every number has an infinite number of invisible zeros to the left of the number...

Denary: 9 is really ...0000000000009

Binary: 1 is really ...0000000000001

...and once you run out of numbers, you add 1 to the next column to the left and reset the current column to 0.

Denary: After 9 comes 10

Binary: After 1 comes 10

After 1011 comes 1100



Binary Number System

Counting is pretty easy, but how do we convert an arbitrary denary number to binary?

Decimal	Binary
0	0
1	1
2	10
3	11
4	100
5	101
6	110
7	111
214	?
?	10110001

And also, how do we convert from binary to denary?



Binary to Denary

Converting from binary to denary is straightforward. Simply lay out each bit of the binary string.

Let's consider the **binary** number 10110001

256	128	64	32	16	8	4	2	1
0	1	0	1	1	0	0	0	1

$$128 + 32 + 16 + 1 = 177$$

Then we get the resulting denary number by simply adding the columns which show a 1.



Denary to Binary

Converting from denary to binary is a little more involved.

We will look at two methods:

Left-to-right subtraction

Repeatedly **subtract** powers of **two** from the number.

Right-to-left division

Repeatedly **divide** the number by **two** and keep track of remainders.



Denary to Binary

Method 1: Left-to-right subtraction.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly subtract powers of two from the number, from left to right, until we reach 0.

256	128	64	32	16	8	4	2	1

214



Denary to Binary

Method 1: Left-to-right subtraction.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly subtract powers of two from the number, from left to right, until we reach 0.

256	128	64	32	16	8	4	2	1

214 is smaller than 256; write 0.



Denary to Binary

Method 1: Left-to-right subtraction.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly subtract powers of two from the number, from left to right, until we reach 0.

256	128	64	32	16	8	4	2	1
0								

214



Denary to Binary

Method 1: Left-to-right subtraction.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly subtract powers of two from the number, from left to right, until we reach 0.

256	128	64	32	16	8	4	2	1
0								

214 - 128 = **86**; write **1**.



Denary to Binary

Method 1: Left-to-right subtraction.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly subtract powers of two from the number, from left to right, until we reach 0.

256	128	64	32	16	8	4	2	1
0	1							

86



Denary to Binary

Method 1: Left-to-right subtraction.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly subtract powers of two from the number, from left to right, until we reach 0.

256	128	64	32	16	8	4	2	1
0	1							

86 - 64 = **22**; write **1**.



Denary to Binary

Method 1: Left-to-right subtraction.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly subtract powers of two from the number, from left to right, until we reach 0.

256	128	64	32	16	8	4	2	1
0	1	1						



Denary to Binary

Method 1: Left-to-right subtraction.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly subtract powers of two from the number, from left to right, until we reach 0.

256	128	64	32	16	8	4	2	1
0	1	1						

22 is smaller than 32; write **0**.



Denary to Binary

Method 1: Left-to-right subtraction.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly subtract powers of two from the number, from left to right, until we reach 0.

256	128	64	32	16	8	4	2	1
0	1	1	0					



Denary to Binary

Method 1: Left-to-right subtraction.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly subtract powers of two from the number, from left to right, until we reach 0.

256	128	64	32	16	8	4	2	1
0	1	1	0					

22 - 16 = **6**; write **1**.



Denary to Binary

Method 1: Left-to-right subtraction.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly subtract powers of two from the number, from left to right, until we reach 0.

256	128	64	32	16	8	4	2	1
0	1	1	0	1				



Denary to Binary

Method 1: Left-to-right subtraction.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly subtract powers of two from the number, from left to right, until we reach 0.

256	128	64	32	16	8	4	2	1
0	1	1	0	1				

6 is smaller than **8**; write **0**.



Denary to Binary

Method 1: Left-to-right subtraction.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly subtract powers of two from the number, from left to right, until we reach 0.

256	128	64	32	16	8	4	2	1
0	1	1	0	1	0			



Denary to Binary

Method 1: Left-to-right subtraction.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly subtract powers of two from the number, from left to right, until we reach 0.

256	128	64	32	16	8	4	2	1
0	1	1	0	1	0			

6 - 4 = **2**; write **1**.



Denary to Binary

Method 1: Left-to-right subtraction.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly subtract powers of two from the number, from left to right, until we reach 0.

256	128	64	32	16	8	4	2	1
0	1	1	0	1	0	1		



Denary to Binary

Method 1: Left-to-right subtraction.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly subtract powers of two from the number, from left to right, until we reach 0.

256	128	64	32	16	8	4	2	1
0	1	1	0	1	0	1		

2 - 2 = **0**; write **1**.



Denary to Binary

Method 1: Left-to-right subtraction.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly subtract powers of two from the number, from left to right, until we reach 0.

256	128	64	32	16	8	4	2	1
0	1	1	0	1	0	1	1	

0



Denary to Binary

Method 1: Left-to-right subtraction.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly subtract powers of two from the number, from left to right, until we reach 0.

256	128	64	32	16	8	4	2	1
0	1	1	0	1	0	1	1	

0 reached; write 0 in all remaining columns.



Denary to Binary

Method 1: Left-to-right subtraction.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly subtract powers of two from the number, from left to right, until we reach 0.

256	128	64	32	16	8	4	2	1
0	1	1	0	1	0	1	1	0

214 in binary is **011010110**



Denary to Binary

Method 2: Right-to-left division.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly divide the number by 2 and write the remainder in the next column.

256	128	64	32	16	8	4	2	1

214



Denary to Binary

Method 2: Right-to-left division.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly divide the number by 2 and write the remainder in the next column.

256	128	64	32	16	8	4	2	1

214 / 2 = 107 remainder 0; write 0.



Denary to Binary

Method 2: Right-to-left division.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly divide the number by 2 and write the remainder in the next column.

256	128	64	32	16	8	4	2	1
								0

107



Denary to Binary

Method 2: Right-to-left division.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly divide the number by 2 and write the remainder in the next column.

256	128	64	32	16	8	4	2	1
								0

107 / 2 = **53** remainder **1**; write **1**.



Denary to Binary

Method 2: Right-to-left division.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly divide the number by 2 and write the remainder in the next column.

256	128	64	32	16	8	4	2	1
							1	0



Denary to Binary

Method 2: Right-to-left division.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly divide the number by 2 and write the remainder in the next column.

256	128	64	32	16	8	4	2	1
							1	0

53 / 2 = **26** remainder **1**; write **1**.



Denary to Binary

Method 2: Right-to-left division.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly divide the number by 2 and write the remainder in the next column.

256	128	64	32	16	8	4	2	1
						1	1	0



Denary to Binary

Method 2: Right-to-left division.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly divide the number by 2 and write the remainder in the next column.

256	128	64	32	16	8	4	2	1
						1	1	0

26 / 2 = 13 remainder 0; write 0.



Denary to Binary

Method 2: Right-to-left division.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly divide the number by 2 and write the remainder in the next column.

256	128	64	32	16	8	4	2	1
					0	1	1	0



Denary to Binary

Method 2: Right-to-left division.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly divide the number by 2 and write the remainder in the next column.

256	128	64	32	16	8	4	2	1
					0	1	1	0

13 / 2 = **6** remainder **1**; write **1**.



Denary to Binary

Method 2: Right-to-left division.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly divide the number by 2 and write the remainder in the next column.

256	128	64	32	16	8	4	2	1
				1	0	1	1	0



Denary to Binary

Method 2: Right-to-left division.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly divide the number by 2 and write the remainder in the next column.

256	128	64	32	16	8	4	2	1
				1	0	1	1	0

6 / 2 = **3** remainder **0**; write **0**.



Denary to Binary

Method 2: Right-to-left division.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly divide the number by 2 and write the remainder in the next column.

256	128	64	32	16	8	4	2	1
			0	1	0	1	1	0



Denary to Binary

Method 2: Right-to-left division.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly divide the number by 2 and write the remainder in the next column.

256	128	64	32	16	8	4	2	1
			0	1	0	1	1	0

3 / 2 = **1** remainder **1**; write **1**.



Denary to Binary

Method 2: Right-to-left division.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly divide the number by 2 and write the remainder in the next column.

256	128	64	32	16	8	4	2	1
		1	0	1	0	1	1	0



Denary to Binary

Method 2: Right-to-left division.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly divide the number by 2 and write the remainder in the next column.

256	128	64	32	16	8	4	2	1
		1	0	1	0	1	1	0

1 / 2 = **0** remainder **1**; write **1**.



Denary to Binary

Method 2: Right-to-left division.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly divide the number by 2 and write the remainder in the next column.

256	128	64	32	16	8	4	2	1
	1	1	0	1	0	1	1	0

0



Denary to Binary

Method 2: Right-to-left division.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly divide the number by 2 and write the remainder in the next column.

256	128	64	32	16	8	4	2	1
	1	1	0	1	0	1	1	0

0 reached; write 0 in all remaining columns.



Denary to Binary

Method 2: Right-to-left division.

Let's consider the *denary* number 214

10^8	10^7	10^6	10^5	10000	1000	100	10	1
0	0	0	0	0	0	2	1	4

To convert to *binary*, we repeatedly divide the number by 2 and write the remainder in the next column.

256	128	64	32	16	8	4	2	1
0	1	1	0	1	0	1	1	0

214 in binary is **11010110**

(the same answer
as Method 1)



Binary Number System

Decimal	Binary
0	0
1	1
2	10
3	11
4	100
5	101
6	110
7	111
214	11010110
177	10110001



03

Representing Colour & Text

Binary is a number system, so it is easy to represent numbers. But how do we represent other kinds of data?



Representing Colour & Text

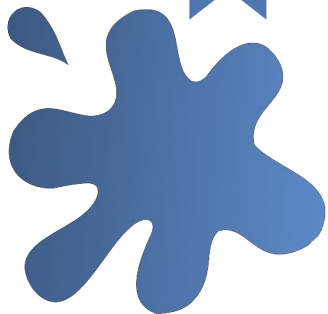
83



S



01010011



In order to *store* and *operate on* everything as *binary* (0s and 1s), we need to be able to **represent** everything as binary.

This means that lots of different items of data (numbers, text, colours, images, videos, etc.) will have a common structure (strings of binary digits).



Representing Colour & Text

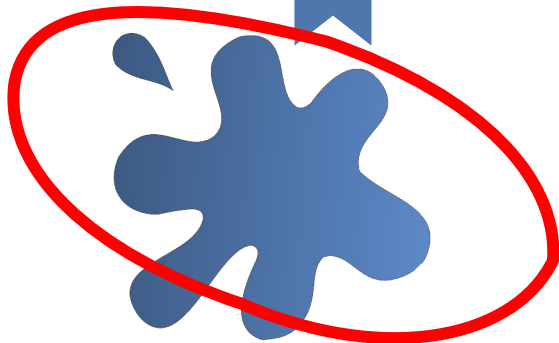
83



S



01010011



We know now how to represent numbers.

(Well, we know how to represent *positive whole numbers*.)

But how could we use binary to represent **colour** or text?



Representing Colour

There are lots of ways we could represent colours in binary.

In order to design our representation, we first have to ask ourselves:

- ❑ Which colours do we want to represent?
- ❑ How many bits can we use to represent a colour?

All of them?

**The human eye
can distinguish
10,000,000
colours**



Representing Colour

There are lots of ways we could represent colours in binary.

In order to design our representation, we first have to ask ourselves:

- ❑ Which colours do we want to represent?
- ❑ How many bits can we use to represent a colour?

**How many bits
would we need
to represent
10,000,000
colours?**

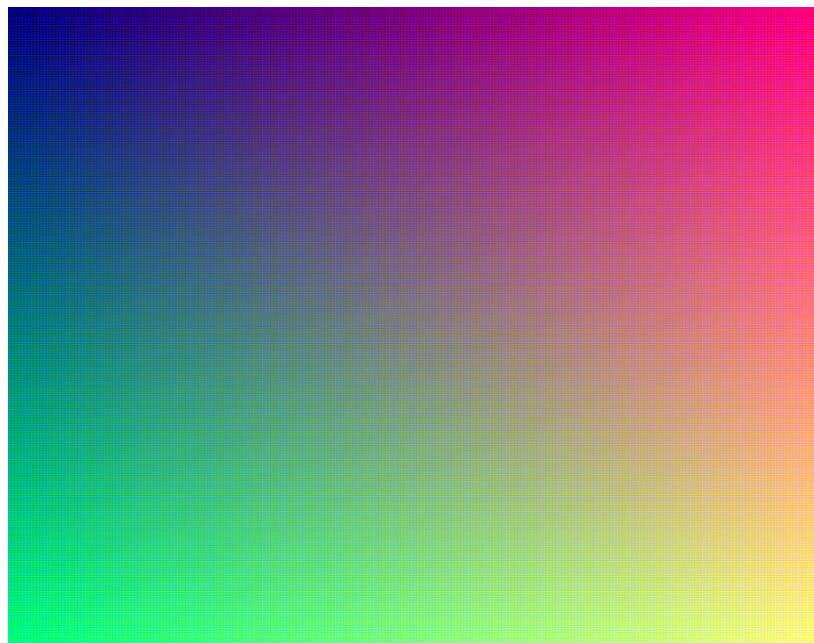


Representing Colour

In order to represent at least 10,000,000 colours, we need 24 bits for each colour.

101100001111001100000101

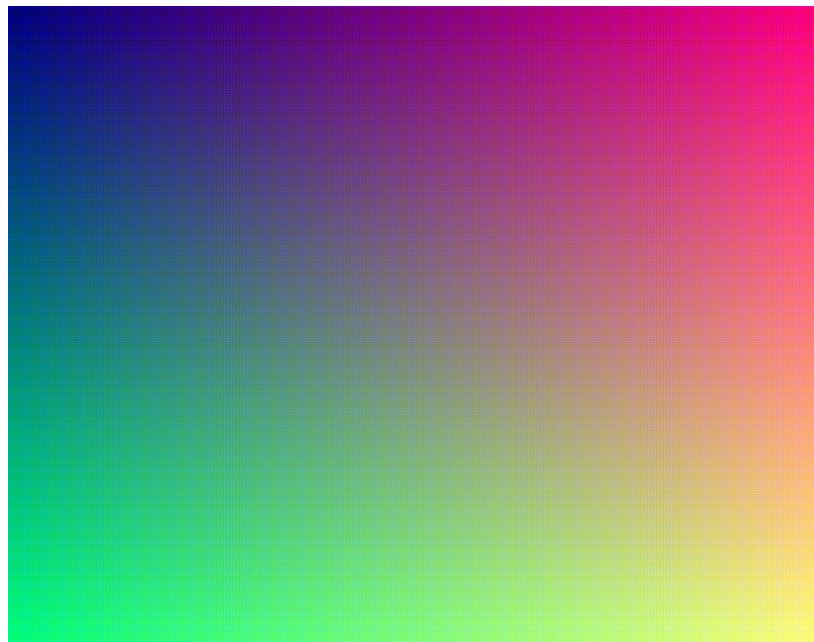
But *how* should a given 24-bit binary word relate to a colour?





Representing Colour

What if we just list all the
colours?





Representing Colour

What if we just list all the colours?

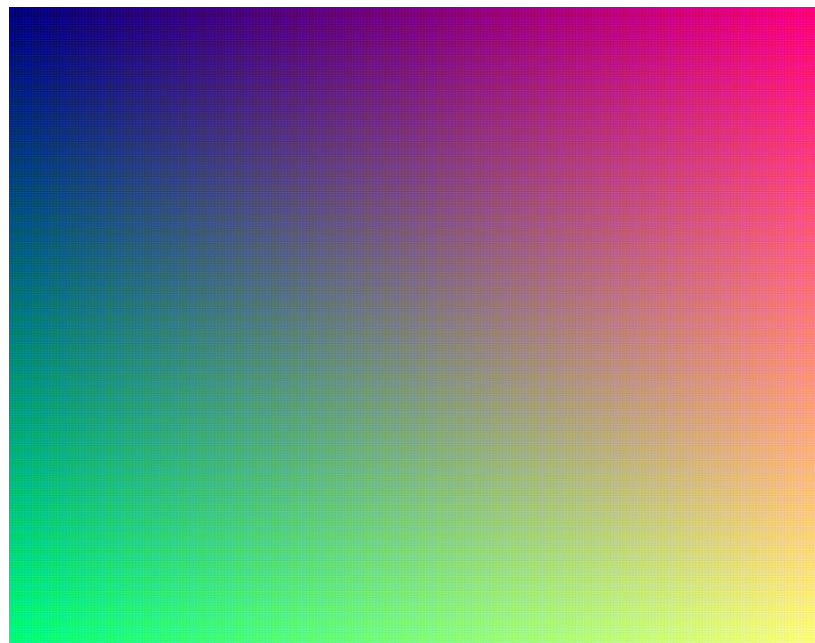
Colour 0 (000000000000000000000000) = **Black**

Colour 1 (000000000000000000000001) = **Green**

...

Colour 11,596,549 (101100001111001100000101)
= **A kind of pinky shade of yellow**

This is a bad representation, because there is no way to work out which colour we mean from a given binary word, and no way to work out what the binary word should be for a given colour.

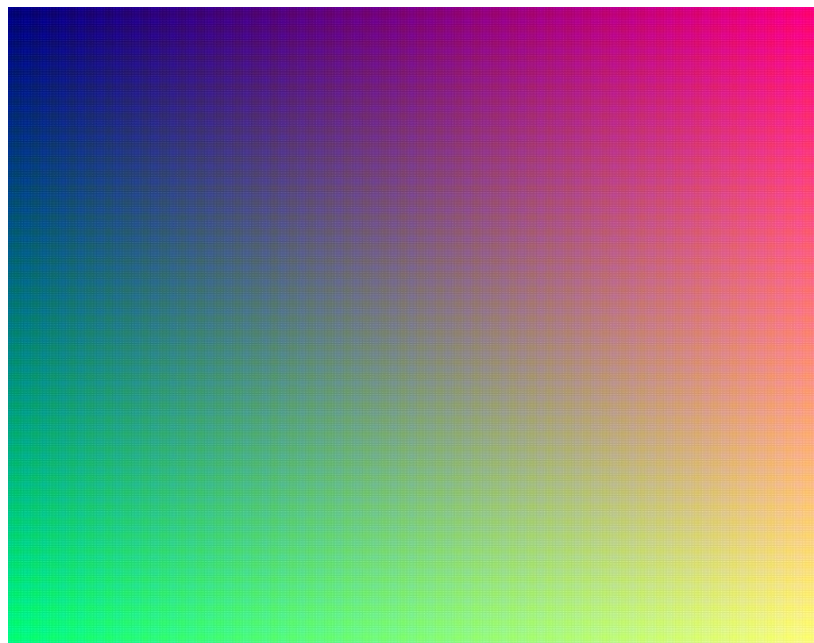




Representing Colour

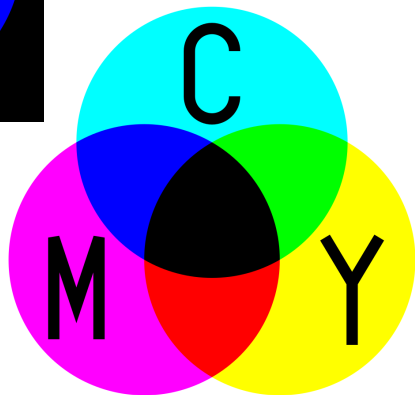
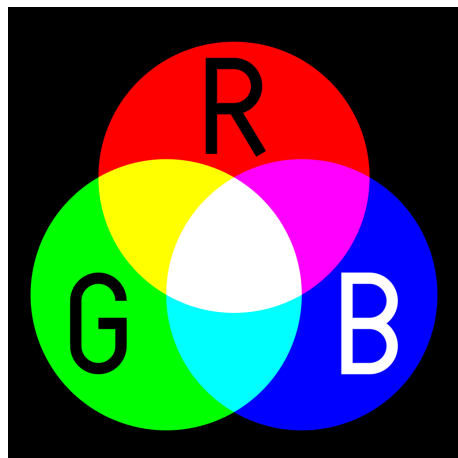
A good representation gives a clear, *mathematical* relationship between the representing type of data (binary) and the represented type of data (colour)

So, let's think about the structure of *colour*, as a piece of data!





Representing Colour



What is a *colour* made up of?

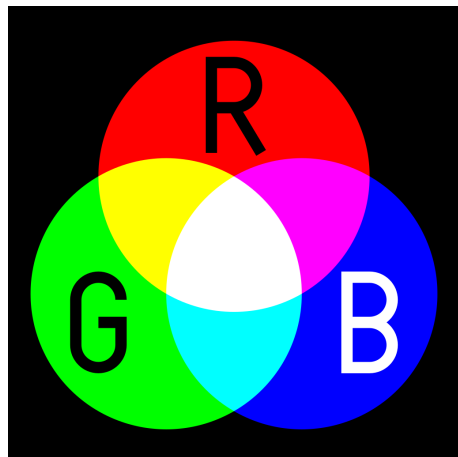
A mixture of primary colours!

In *additive* (RGB) colour models, every colour can be modelled as a mixture of **red**, **green** and **blue**, which is *added* to **black**.

In *subtractive* (CMY) colour models, every colour can be modelled as a mixture of **cyan**, **magenta** and **yellow**, which is *subtracted* from **white**.



Representing Colour



RGB colour models make slightly more sense for a computer... *why?*

Because computer displays are (usually) **black**.

So we will model each colour as a mixture of **red**, **green** and **blue**.



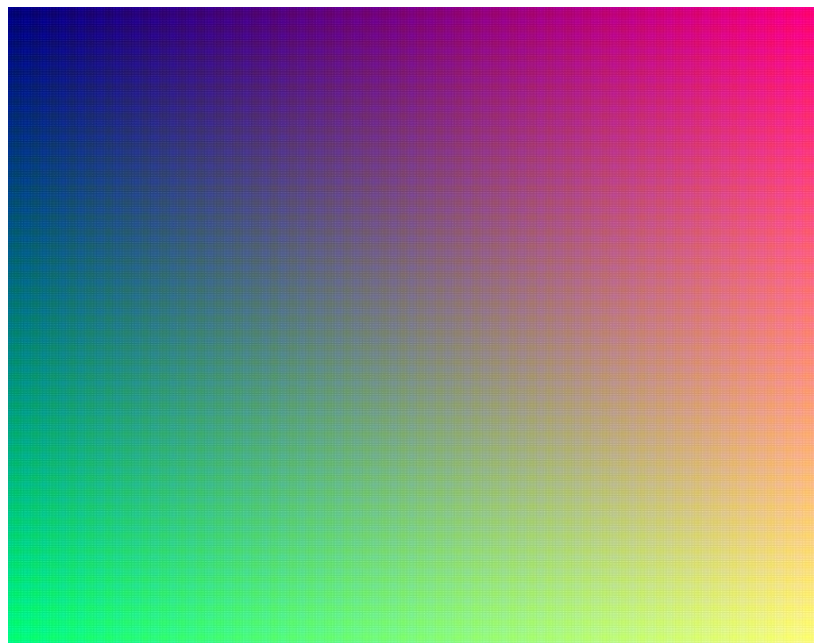
Representing Colour

10110000 11110011 00000101

So *how* should a given 24-bit binary word relate to a colour, which is a mixture of **red**, **green** and **blue**?

Let's split the 24 bits into three bytes.

The first byte represents **red**, the next represents **green** and the last represents **blue**.





Representing Colour

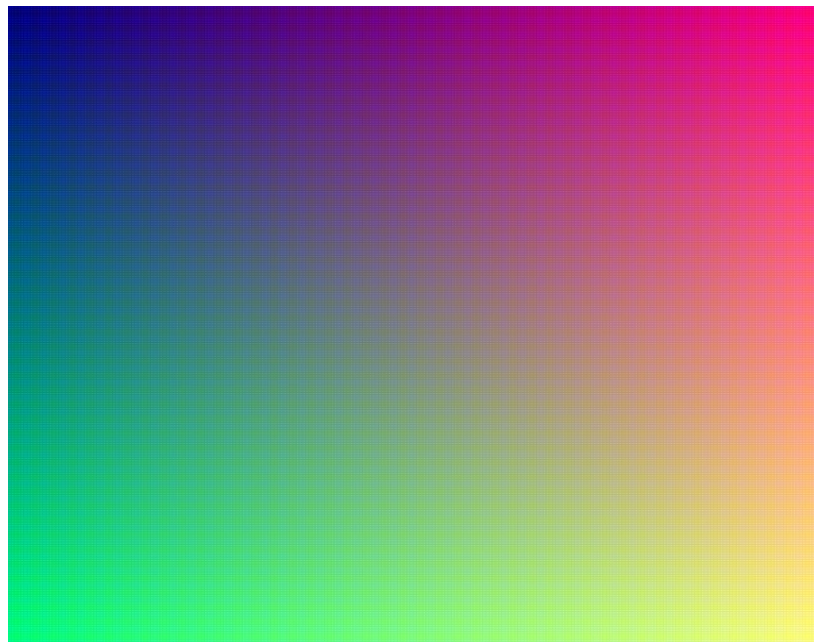
10110000 11110011 00000101

Recap: How many possible values are there for a byte? 256

What is the minimum value? 0

What is the maximum value? 255

Therefore, each byte will measure the *amount* of that colour on a scale from 0-255.





Representing Colour

$(10110000)_2$ $(11110011)_2$ $(00000101)_2$

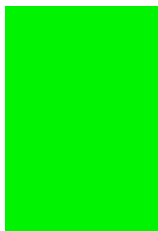
$(176)_{10}$

$(243)_{10}$

$(5)_{10}$



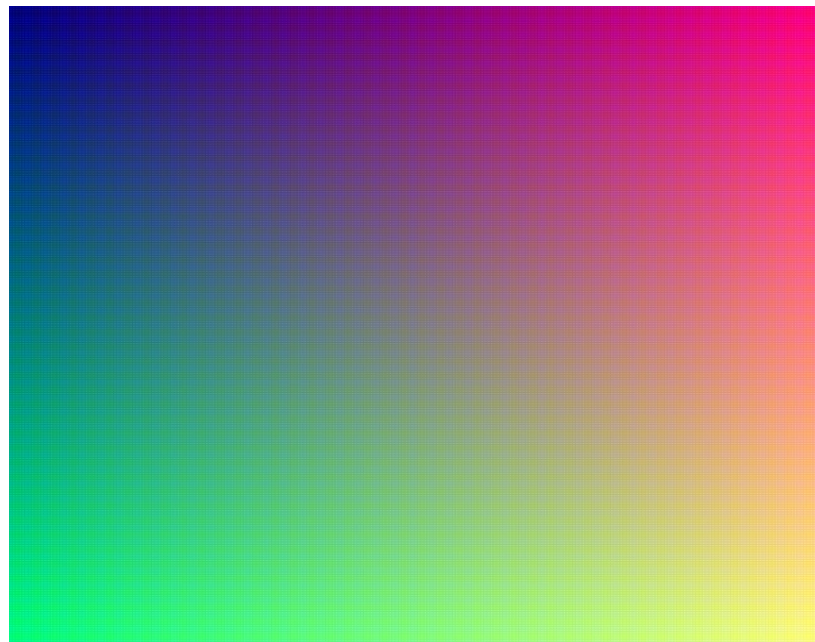
+



+



=



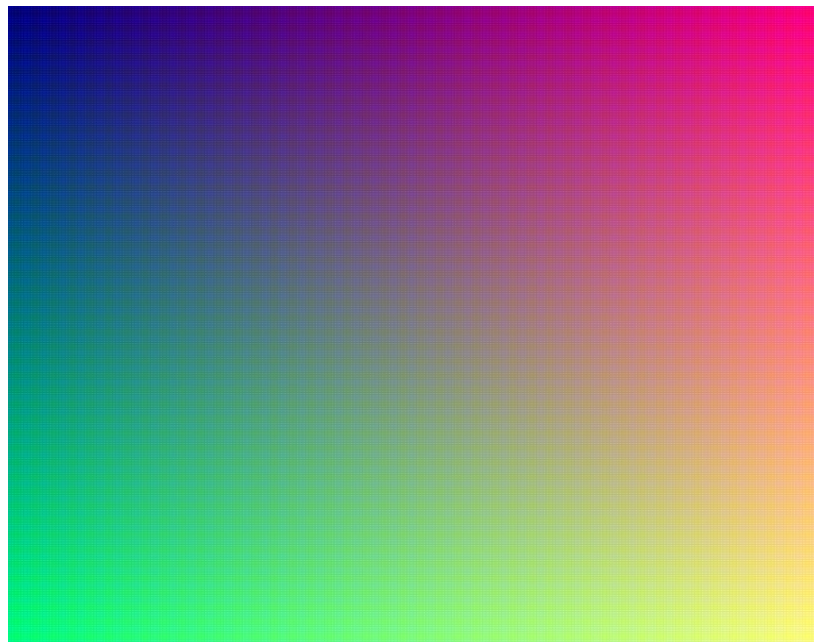


Representing Colour

$(10110000)_2$ $(11110011)_2$ $(00000101)_2$
↓ ↓ ↓
 $(176)_{10}$ $(243)_{10}$ $(5)_{10}$

Using this 24-bit colour representation, we can write this colour in many different ways:

In **binary** (base 2 number system)
it's **101100001111001100000101**.



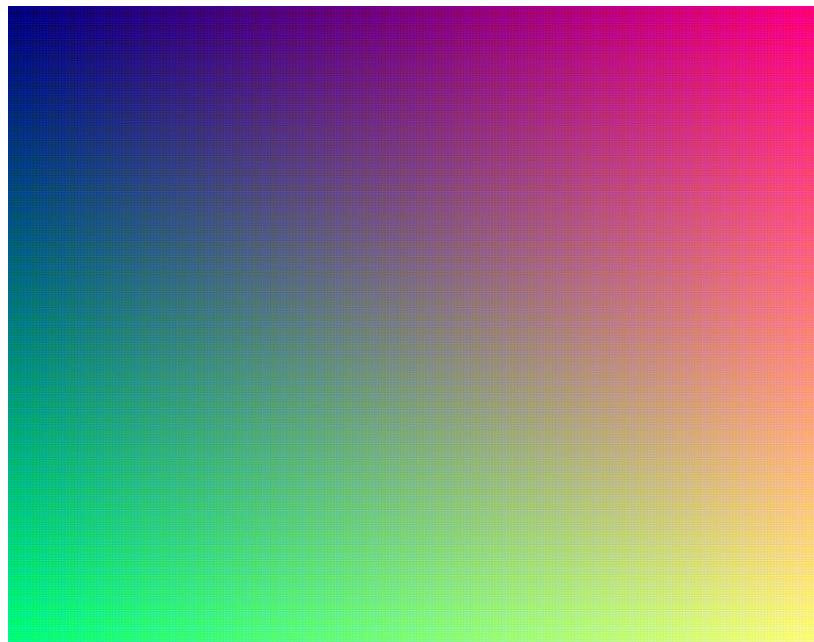


Representing Colour

$(10110000)_2$ $(11110011)_2$ $(00000101)_2$
↓ ↓ ↓
 $(176)_{10}$ $(243)_{10}$ $(5)_{10}$

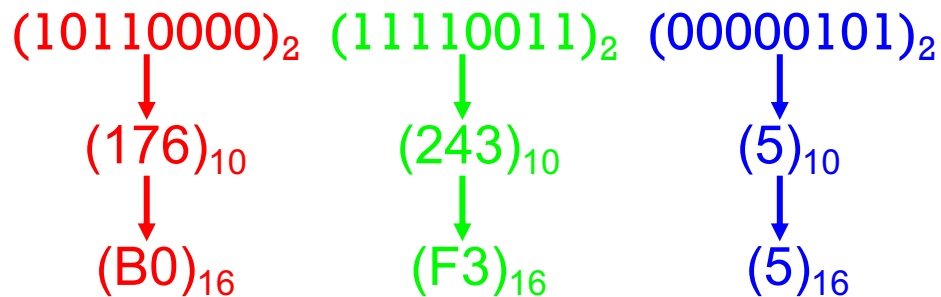
Using this 24-bit colour representation, we can write this colour in many different ways:

In RGB notation, which uses **denary** (base 10 number system) it's **rgb(176,243,5)**.





Representing Colour



As a hex code, which uses **hexadecimal** (base 16 number system) it's **#B0F305**.

Hexadecimal uses 16 distinct symbols (0-9, A-F). Therefore, every 4-bit nibble relates to a hex character:

0000	0	1000	8
0001	1	1001	9
0010	2	1010	A
0011	3	1011	B
0100	4	1100	C
0101	5	1101	D
0110	6	1110	E
0111	7	1111	F



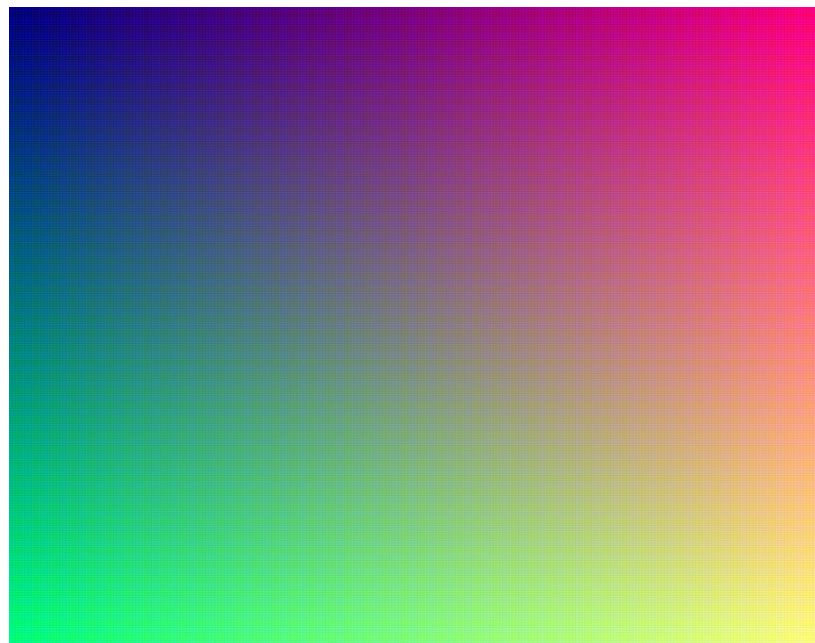
Representing Colour

Memory on a computer is finite.

We can represent every colour we need using 24-bits, but what if we have fewer bits than that?

While 24, 32 and 64-bit binary words are commonly used today, this hasn't always been the case.

Older games consoles only had 16, 8 or even 2-bits for their binary words. What did this mean for the colours of their games?





Representing Colour

Simple: with fewer bits, we have fewer colours.

For example, with 8-bits there are only 256 different binary words: so we can only have 256 colours.

VGA display adapters, released in '87, often used 256 colours.

Before that, EGA used 16 colours.

The Game Boy had only 2 bits for its colours.





Here's *The Secret of Monkey Island* in 1-bit, 2-bit, 4-bit and 8-bit colour.

(source: Point and Click Jam!)



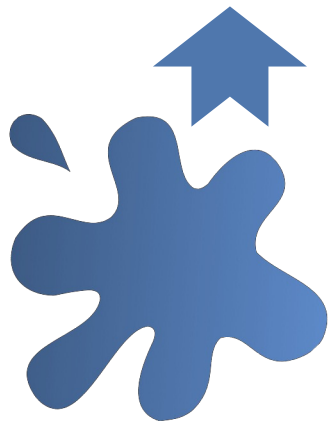


Representing Colour & Text

83



01010011



We know now how to represent
(positive whole) numbers and
colours.

But how could we use binary to
represent colour or **text**?

Representing Text

There are lots of ways we could represent text in binary.

All of them?

As we learned with numbers and colours, the number of bits we have is directly related to the number of items (in this case, textual **characters**) we can represent.

So, which characters do we want to be able to store?

Representing Text

a ª À Á Â Ã Ä Å à á â ã ä å Ä
 ā Ă ă Ą ą Ą ă Ą ā Ą ā Ą ą Ą Ą
 Â â Ą à ˆ ˆ ˆ ˆ ˆ ˆ ˆ ˆ ˆ ˆ ˆ ˆ
 à Ą ă Ą ă Ą Ą Ą ă Ą à Ą ă Ą ă
 Ǻ ǻ Ǻ (A) (a) Ⓐ ⓐ Ⓐ ⓐ Ⓐ ⓐ Ⓐ ⓐ Ⓐ ⓐ
 a A a A a A a A A a A a A a

圖	族	人	行	事	後	先	答	品	秋	惡	前	始	大	鳥	黑
DRAWING	FAMILY	PERSON	ACT	AFFAIR	AFTER	AHEAD	ANSWER	ARTICLE	AUTUMN	BAD	BEFORE	BEGIN	BIG	BIRD	BLACK
青	台	體	本	借	域	明	建	館	買	運	因	字	長	子	漢
BLUE	BOARD	BODY	BOOK	BORROW	BOUNDED	BRIGHT	BUILD	BUILDING	BUY	CARRY	CAUSE	CHARACTER	CHIEF	CHILD	CHINESE
衣	服	雲	集	來	委	社	寫	國	工	習	切	暗	曜	度	死
CLOTHES	CLOTHING	CLOUD	COLLECT	COME	COMMIT	COMPANY	COPY	COUNTRY	CRAFT	CUSTOM	CUT	DARK	DAY	DEGREE	DIE
方	醫	犬	下	引	画	飲	早	東	食	電	終	勉	入	用	永
DIRECTION	DOCTOR	DOG	DOWN	DRAW	DRAWING	DRINK	EARLY	EAST	EAT	ELECTRICITY	END	ENDEAVOUR	ENTER	ERRAND	ETERNAL
夕	每	高	目	家	父	羽	料	野	火	肉	花	足	友	以	榮
EVENING	EVERY	EXPENSIVE	EYE	FAMILY	FATHER	FEATHER	FEE	FIELD	FIRE	FLESH	FLOWER	FOOT	FRIEND	FROM	GLORY
午	金	地	半	堂	手	有	聞	心	天	重	持	屋	百	病	中
GO	GOLD	GROUND	HALF	HALL	HAND	HAVE	HEAR	HEART	HEAVENS	HEAVY	HOLD	HOUSE	HUNDRED	ILLNESS	IN

All of them?

There are tens of
thousands of unique
characters across the
world.



Representing Text

a ª À Á Â Ã Ä Å à á â ã ä å Ä
 ā Ă ă Ą ą Ą ă Ą ā Ą ā Ą ă Ą ă
 Â â Ą à ˆ ˆ ˆ ˆ ˆ ˆ ˆ ˆ ˆ ˆ ˆ ˆ
 à ă ă ă ă ă ă ă ă ă ă ă ă ă ă
 ă ă ă ă ă ă ă ă ă ă ă ă ă ă
 ă ă ă ă ă ă ă ă ă ă ă ă ă ă
 a A a A a A a A A a A a A a A

圖	族	人	行	事	後	先	答	品	秋	惡	前	始	大	鳥	黑
DRAWING	FAMILY	PERSON	ACT	AFFAIR	AFTER	AHEAD	ANSWER	ARTICLE	AUTUMN	BAD	BEFORE	BEGIN	BIG	BIRD	BLACK
青	台	體	本	借	域	明	建	館	買	運	因	字	長	子	漢
BLUE	BOARD	BODY	BOOK	BORROW	BOUNDED	BRIGHT	BUILD	BUILDING	BUY	CARRY	CAUSE	CHARACTER	CHIEF	CHILD	CHINESE
衣	服	雲	集	來	委	社	寫	國	工	習	切	暗	曜	度	死
CLOTHES	CLOTHING	CLOUD	COLLECT	COME	COMMIT	COMPANY	COPY	COUNTRY	CRAFT	CUSTOM	CUT	DARK	DAY	DEGREE	DIE
方	醫	犬	下	引	畫	飲	早	東	食	電	終	勉	入	用	永
DIRECTION	DOCTOR	DOG	DOWN	DRAW	DRAWING	DRINK	EARLY	EAST	EAT	ELECTRICITY	END	ENDEAVOUR	ENTER	ERRAND	ETERNAL
夕	每	高	目	家	父	羽	料	野	火	肉	花	足	友	以	榮
EVENING	EVERY	EXPENSIVE	EYE	FAMILY	FATHER	FEATHER	FEE	FIELD	FIRE	FLESH	FLOWER	FOOT	FRIEND	FROM	GLORY
午	金	地	半	堂	手	有	聞	心	天	重	持	屋	百	病	中
GO	GOLD	GROUND	HALF	HALL	HAND	HAVE	HEAR	HEART	HEAVENS	HEAVY	HOLD	HOUSE	HUNDRED	ILLNESS	IN

But, unlike with colour, it might be easier for us to *settle for less* with text.

Let's say we only need to display English text. How many bits do we *actually need*?



Representing Text

(No Model.)

11 Sheets—Sheet 6.

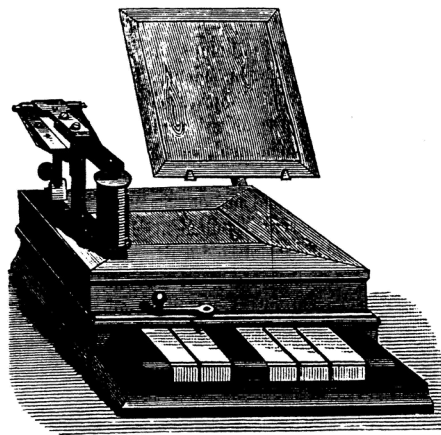
J. M. E. BAUDOT.

PRINTING TELEGRAPH.

No. 388,244.

Patented Aug. 21, 1888.

Fig. 2A.



	1	2	3	4	5
A	+	-	-	-	-
B	+	-	+	+	-
C	+	+	+	+	-
D	+	+	+	+	-
E	+	+	+	+	-
F	+	+	+	+	-
G	+	+	+	+	-
H	+	+	+	+	-
I	+	+	+	+	-
J	+	+	+	+	-
K	+	+	+	+	-
L	+	+	+	+	-
M	+	+	+	+	-
N	+	+	+	+	-
O	+	+	+	+	-
P	+	+	+	+	-
Q	+	+	+	+	-
R	+	+	+	+	-
S	+	+	+	+	-
T	+	+	+	+	-
U	+	+	+	+	-
V	+	+	+	+	-
W	+	+	+	+	-
X	+	+	+	+	-
Y	+	+	+	+	-
Z	+	+	+	+	-
Blank	+	+	+	+	-

The Baudot code, an early telegraphic system, used a 5-bit representation for characters, meaning a maximum of $2^5 = 32$ characters.

These 32 'codes' allowed one to represent (or 'encode') the 26 letters, as well as two symbols and four ways of writing a blank.

INVENTOR:

Jean Maurice Emile Baudot

Representing Text

00	01	02	03	04	05	06	07
NUL	E 3	LF	A -	SP	S ' 8	I	U 7
08	09	0A	0B	0C	0D	0E	0F
CR	D ENQ	R 4	J BEL	N ,	F !	C :	K <
10	11	12	13	14	15	16	17
T 5	Z +	L >	W 2	H £	Y 6	P 0	Q 1
18	19	1A	1B	1C	1D	1E	1F
O 9	B ?	G &	FIGS	M .	X /	U ;	LTRS
Letters		Figures		Control Chars.			

In 1932, the ITA2 code was introduced, which had 5-bits and a shift key. With a maximum of 64 different characters, all common letters, numbers and symbols could be represented.

Are there any problems with this representation?



Representing Text

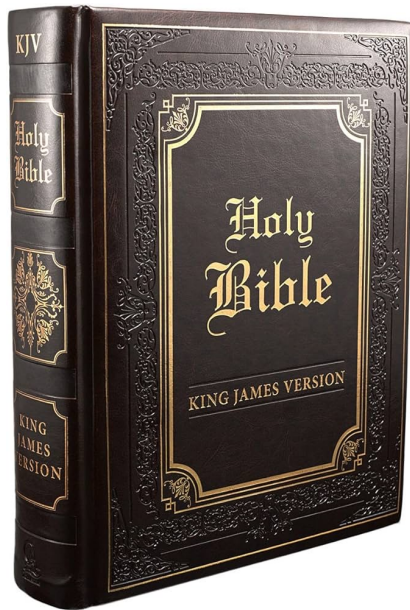
Bits					Column							
b ₇ b ₆ b ₅ b ₄ b ₃ b ₂ b ₁					Row							
b ₄	b ₃	b ₂	b ₁		0	1	2	3	4	5	6	7
0	0	0	0	0	NUL	DLE	SP	0	@	P	`	p
0	0	0	0	1	SOH	DC1	!	1	A	Q	a	q
0	0	0	1	0	2	STX	DC2	"	2	B	R	b
0	0	0	1	1	3	ETX	DC3	#	3	C	S	s
0	1	0	0	0	4	EOT	DC4	\$	4	D	T	t
0	1	0	0	1	5	ENQ	NAK	%	5	E	U	u
0	1	0	1	0	6	ACK	SYN	&	6	F	V	v
0	1	0	1	1	7	BEL	ETB	'	7	G	W	w
1	0	0	0	0	8	BS	CAN	(	8	H	X	x
1	0	0	0	1	9	HT	EM	)	9	I	Y	y
1	0	0	1	0	10	LF	SUB	*	:	J	Z	z
1	0	0	1	1	11	VT	ESC	+	;	K	[	{
1	0	1	0	0	12	FF	FS	,	<	L	\	
1	0	1	0	1	13	CR	GS	-	=	M	]	}
1	0	1	1	0	14	SO	RS	.	>	N	^	~
1	0	1	1	1	15	SI	US	/	?	O	_	DEL

64 symbols isn't quite enough if we want both uppercase *and* lowercase letters, as well as numbers and common symbols.

So, when computers came along, the standard became a 7-bit system called American Standard Code for Information Interchange (or, ASCII).

Representing Text

**788,258
words**



=

**3,116,480
characters**

ASCII is still used today, and using only 7 bits means that text files remain *very* lightweight.

8 bits in a byte,
1000 bytes in a Kilobyte,
1000 Kilobytes in a Megabyte.

**21,815,360
bits**

= **2.73 MB**



Representing Text

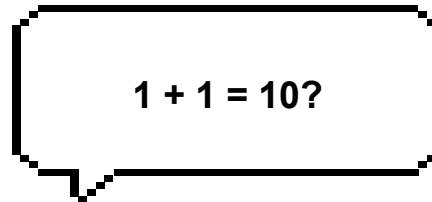
Emoticons (U+1F600 to U+1F64F)

128512	128513	128514	128515	128516	128517	128518	128519	128520	128521	128522	128523
128524	128525	128526	128527	128528	128529	128530	128531	128532	128533	128534	128535
128536	128537	128538	128539	128540	128541	128542	128543	128544	128545	128546	128547
128548	128549	128550	128551	128552	128553	128554	128555	128556	128557	128558	128559
128560	128561	128562	128563	128564	128565	128566	128567	128568	128569	128570	128571
128572	128573	128574	128575	128576	128577	128578	128579	128580	128581	128582	128583
128584	128585	128586	128587	128588	128589	128590	128591				

Sometimes, we need to be able to use more than 128 characters. For example, there are thousands of unique Chinese characters.

For this, the 1988 standard of Unicode was developed. 8-bit Unicode (UTF-8) has been the standard of the World Wide Web since 2008.

UTF-16 and UTF-32 allow us to write even more characters! 🤖



04

Binary Arithmetic

We know how to store binary numbers as different kinds of data. But how do we perform **arithmetic operations** on them?

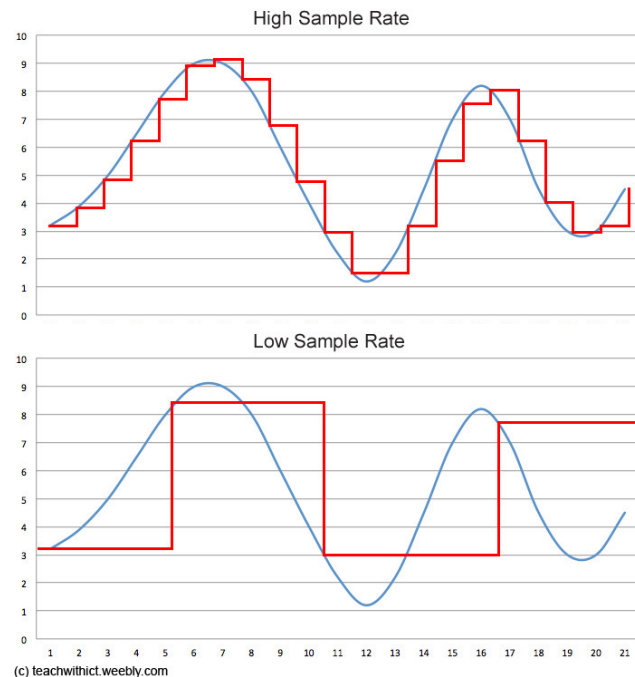


Quick Recap

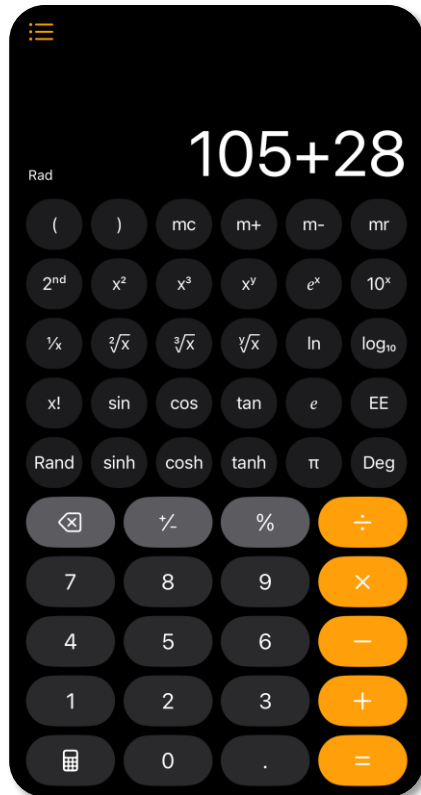
We've introduced the binary number system, and seen how binary can be used to represent colours and text.

From this, you can imagine how binary can be used to store all sorts of data:

- Images are made up of thousands of blocks (called pixels) of colour,
- Audio is represented by a stream of bits that give the amplitude of a digitally sampled sound wave over time,
- Video is a combination of a sequence of images and audio.



Binary Arithmetic



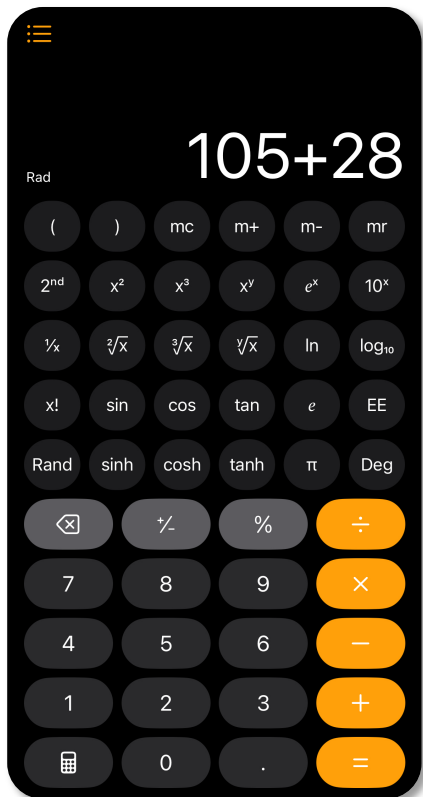
Now let's return to how computers represent **numbers**.

Numbers are used everywhere on a computer, and it's not enough to just *store* numbers – we also need to be able to *operate on* them by performing logical and **arithmetic** operations.

How would a computer, such as a smartphone, *compute* **105 + 28**, given that they are represented in binary?



Binary Arithmetic – Addition



To answer this question, let's think: how would we add **105** and **28** in *denary*?

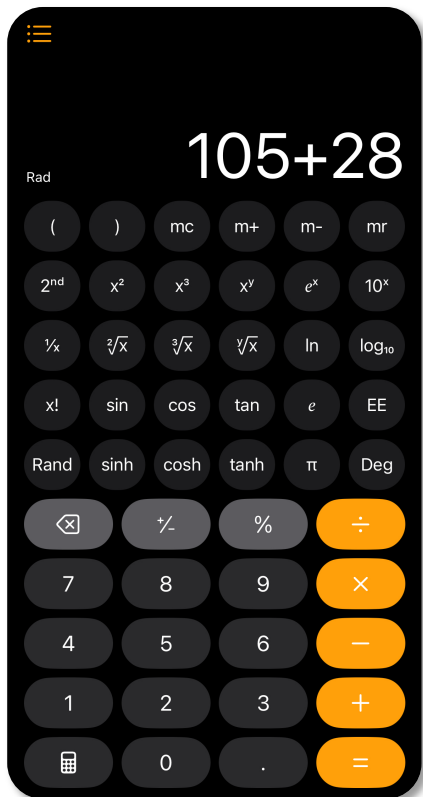
We would write them out...

$$\begin{array}{r} 105 \\ + 28 \\ \hline 133 \end{array}$$

...and add them digit-by-digit (remembering to carry).



Binary Arithmetic – Addition



In order to do this, we had to know how to **add digits** themselves.

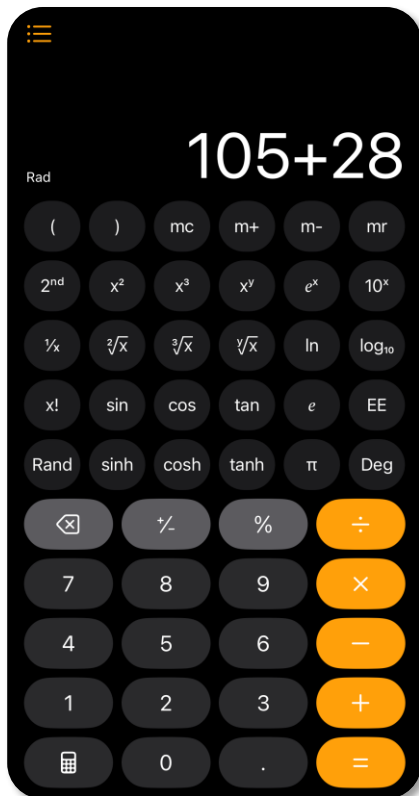
We needed to know that $2 + 1 = 3$...

$$\begin{array}{r} 105 \\ + 28 \\ \hline 133 \end{array}$$

...or that $5 + 8 = 13$.



Binary Arithmetic – Addition

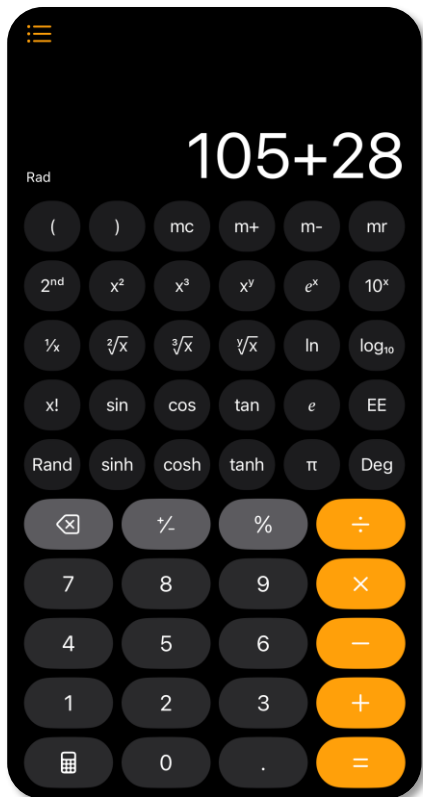


In order to perform binary addition we first need to know how to add individual **bits**.

And, as with denary additionary, **binary addition** has to be performed from right-to-left to take account of carry bits.



Binary Arithmetic – Addition



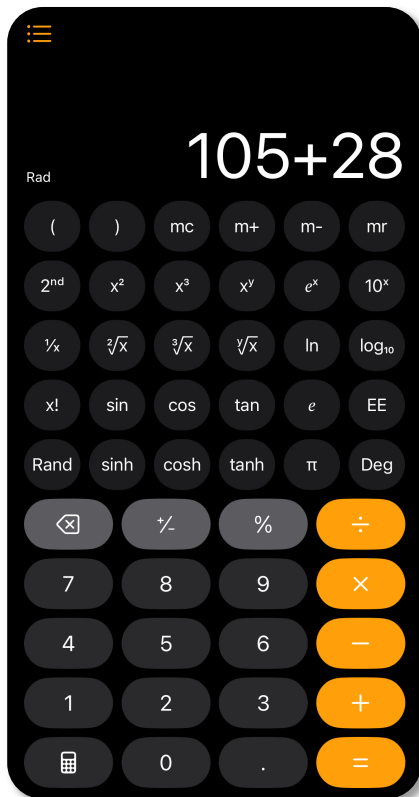
Binary addition has to be performed from right-to-left to take account of carry bits.

A	B	A + B	
		Result	Carry
0	0	0	0
1	0	1	0
0	1	1	0
1	1		

The columns with a zero are easy: we don't need to carry anything.



Binary Arithmetic – Addition



How do we add **1 + 1** in binary?

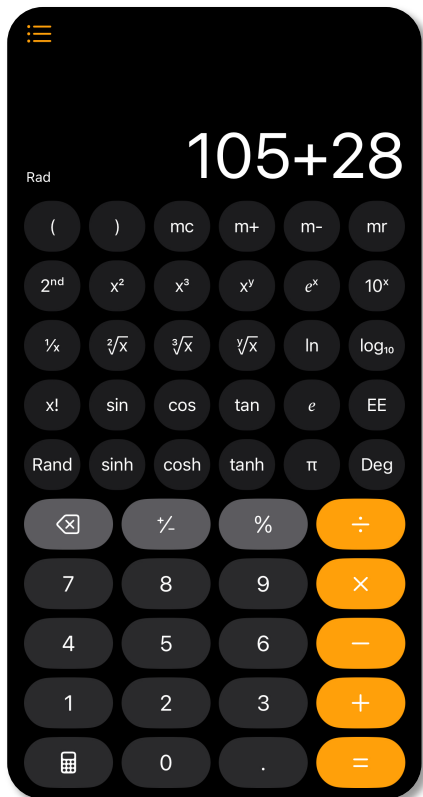
Here, 1 is behaving as both the highest digit and the lowest digit.

A	B	A + B	
		Result	Carry
0	0	0	0
1	0	1	0
0	1	1	0
1	1		

It's a bit like if we wrote $9 + 1$ in *denary*.



Binary Arithmetic – Addition



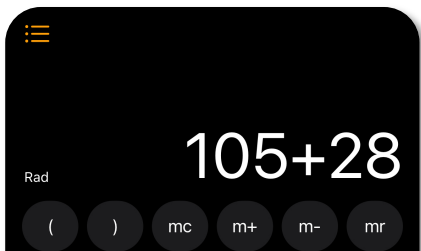
As with $9 + 1$ in *denary*, we need to carry 1 to the next column, and reduce this column to 0.

A	B	A + B	
		Result	Carry
0	0	0	0
1	0	1	0
0	1	1	0
1	1	0	1

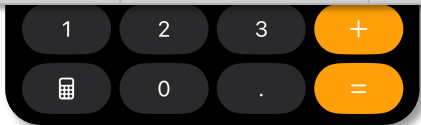
So, $1 + 1 = 10$, or 0 carry 1.



Binary Arithmetic – Addition



A	B	A + B	
		Result	Carry
0	0	0	0
1	0	1	0
0	1	1	0
1	1	0	1



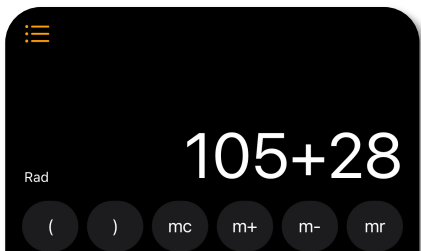
So, to compute **105 + 28** in *binary*, let's first convert 105 and 28 from *denary*...

$$\begin{array}{r} 1101001 \\ + 11100 \\ \hline \end{array}$$

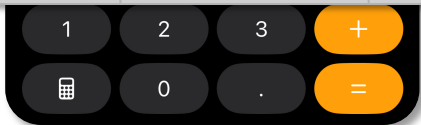
...and then we add bit-by-bit (remembering to carry).



Binary Arithmetic – Addition



A	B	A + B	
		Result	Carry
0	0	0	0
1	0	1	0
0	1	1	0
1	1	0	1



So, to compute **105 + 28** in *binary*,
let's first convert 105 and 28
from *denary*...

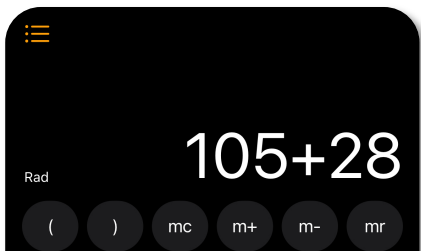
$$\begin{array}{r} 1101001 \\ + 11100 \\ \hline \end{array}$$

The final result of the addition, 1, is highlighted in a red box.

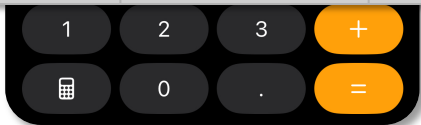
...and then we add bit-by-bit
(remembering to carry).



Binary Arithmetic – Addition



A	B	A + B	
		Result	Carry
0	0	0	0
1	0	1	0
0	1	1	0
1	1	0	1



So, to compute **105 + 28** in *binary*,
let's first convert 105 and 28
from *denary*...

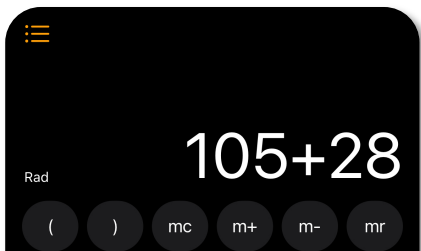
$$\begin{array}{r} 1101001 \\ + 11100 \\ \hline \end{array}$$

The result of the addition is 10001001, with a carry of 1 from the 7th bit position. The carry is highlighted in red.

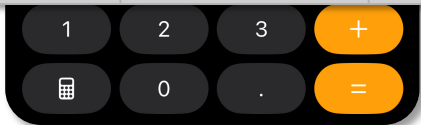
...and then we add bit-by-bit
(remembering to carry).



Binary Arithmetic – Addition



A	B	A + B	
		Result	Carry
0	0	0	0
1	0	1	0
0	1	1	0
1	1	0	1



So, to compute **105 + 28** in *binary*, let's first convert 105 and 28 from *denary*...

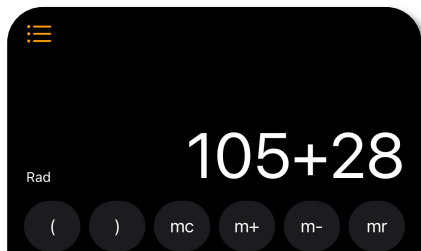
$$\begin{array}{r} 1101001 \\ + 11100 \\ \hline \end{array}$$

01

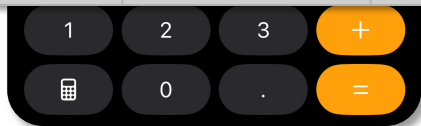
...and then we add bit-by-bit (remembering to carry).



Binary Arithmetic – Addition



A	B	A + B	
		Result	Carry
0	0	0	0
1	0	1	0
0	1	1	0
1	1	0	1



So, to compute **105 + 28** in *binary*, let's first convert 105 and 28 from *denary*...

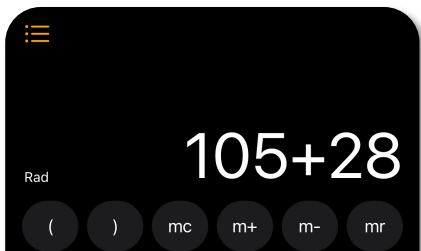
$$\begin{array}{r} 1101001 \\ + 11100 \\ \hline \end{array}$$

The result of the addition is 1010101, with the final carry of 1 added to the right.

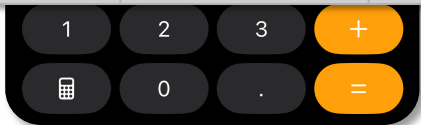
...and then we add bit-by-bit (remembering to carry).



Binary Arithmetic – Addition



A	B	A + B	
		Result	Carry
0	0	0	0
1	0	1	0
0	1	1	0
1	1	0	1



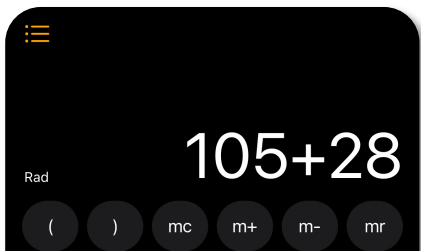
So, to compute **105 + 28** in *binary*, let's first convert 105 and 28 from *denary*...

$$\begin{array}{r} 1101001 \\ + 11100 \\ \hline 101 \end{array}$$

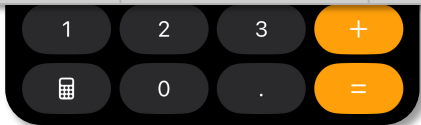
...and then we add bit-by-bit (remembering to carry).



Binary Arithmetic – Addition



A	B	A + B	
		Result	Carry
0	0	0	0
1	0	1	0
0	1	1	0
1	1	0	1



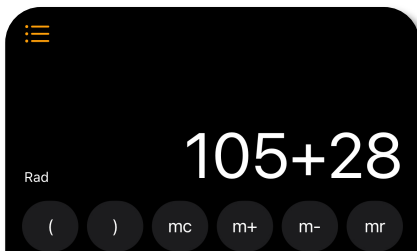
So, to compute **105 + 28** in *binary*, let's first convert 105 and 28 from *denary*...

$$\begin{array}{r} 1101001 \\ + 11100 \\ \hline 101 \end{array}$$

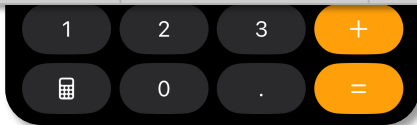
...and then we add bit-by-bit (remembering to carry).



Binary Arithmetic – Addition



A	B	A + B	
		Result	Carry
0	0	0	0
1	0	1	0
0	1	1	0
1	1	0	1



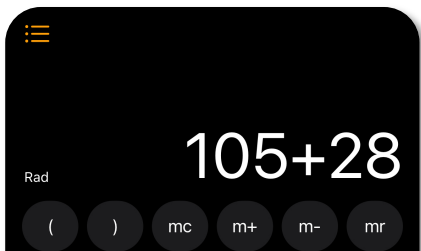
So, to compute **105 + 28** in *binary*, let's first convert 105 and 28 from *denary*...

$$\begin{array}{r} 1101001 \\ + 11100 \\ \hline 10101 \end{array}$$

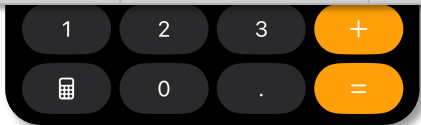
...and then we add bit-by-bit (remembering to carry).



Binary Arithmetic – Addition



A	B	A + B	
		Result	Carry
0	0	0	0
1	0	1	0
0	1	1	0
1	1	0	1



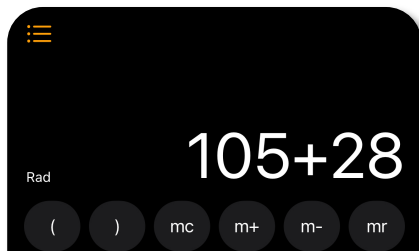
So, to compute **105 + 28** in *binary*, let's first convert 105 and 28 from *denary*...

$$\begin{array}{r} 1101001 \\ + 11100 \\ \hline 10101 \end{array}$$

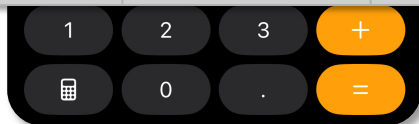
...and then we add bit-by-bit (remembering to carry).



Binary Arithmetic – Addition



A	B	A + B	
		Result	Carry
0	0	0	0
1	0	1	0
0	1	1	0
1	1	0	1



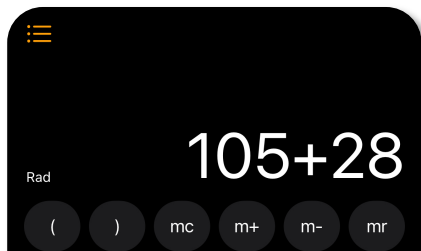
So, to compute **105 + 28** in *binary*, let's first convert 105 and 28 from *denary*...

$$\begin{array}{r} 1101001 \\ + 11100 \\ \hline 100101 \end{array}$$

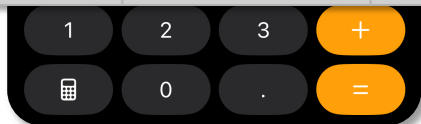
...and then we add bit-by-bit (remembering to carry).



Binary Arithmetic – Addition



A	B	A + B	
		Result	Carry
0	0	0	0
1	0	1	0
0	1	1	0
1	1	0	1



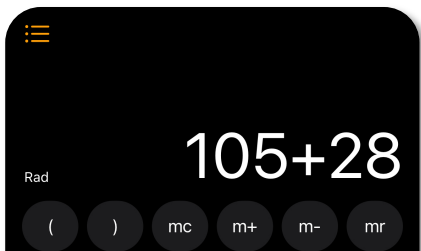
So, to compute **105 + 28** in *binary*, let's first convert 105 and 28 from *denary*...

$$\begin{array}{r} 1101001 \\ + 11100 \\ \hline 100101 \end{array}$$

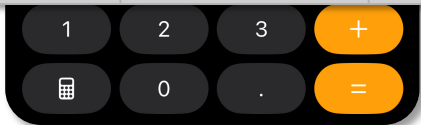
...and then we add bit-by-bit (remembering to carry).



Binary Arithmetic – Addition



A	B	A + B	
		Result	Carry
0	0	0	0
1	0	1	0
0	1	1	0
1	1	0	1



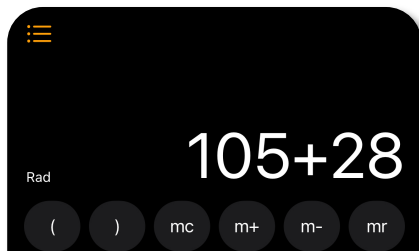
So, to compute **105 + 28** in *binary*, let's first convert 105 and 28 from *denary*...

$$\begin{array}{r} 1101001 \\ + 11100 \\ \hline 1000101 \end{array}$$

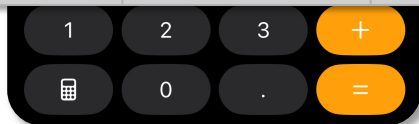
...and then we add bit-by-bit (remembering to carry).



Binary Arithmetic – Addition



A	B	A + B	
		Result	Carry
0	0	0	0
1	0	1	0
0	1	1	0
1	1	0	1



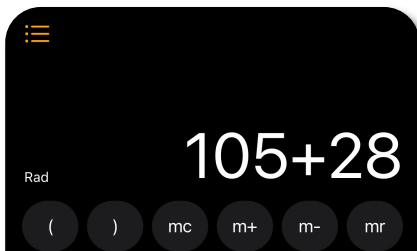
So, to compute **105 + 28** in *binary*, let's first convert 105 and 28 from *denary*...

$$\begin{array}{r} 1101001 \\ + 11100 \\ \hline 1000101 \end{array}$$

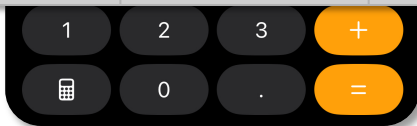
...and then we add bit-by-bit (remembering to carry).



Binary Arithmetic – Addition



A	B	A + B	
		Result	Carry
0	0	0	0
1	0	1	0
0	1	1	0
1	1	0	1



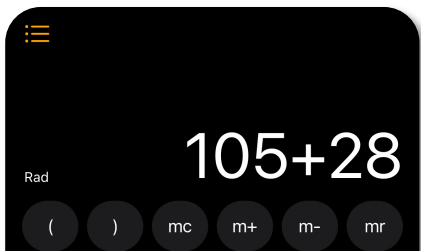
So, to compute **105 + 28** in *binary*,
let's first convert 105 and 28
from *denary*...

$$\begin{array}{r} 1101001 \\ + 11100 \\ \hline 10000101 \end{array}$$

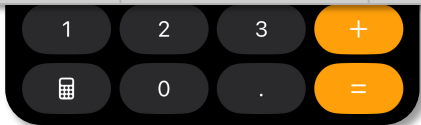
...and then we add bit-by-bit
(remembering to carry).



Binary Arithmetic – Addition



A	B	A + B	
		Result	Carry
0	0	0	0
1	0	1	0
0	1	1	0
1	1	0	1



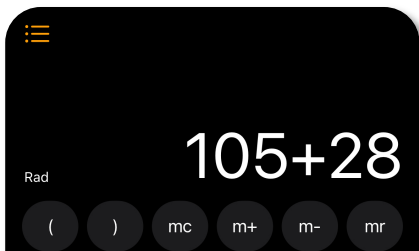
So, to compute **105 + 28** in *binary*, let's first convert 105 and 28 from *denary*...

$$\begin{array}{r} 1101001 \\ + 11100 \\ \hline 10000101 \end{array}$$

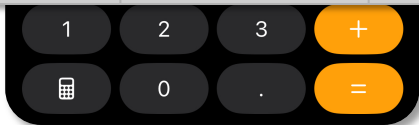
...and then we add bit-by-bit (remembering to carry).



Binary Arithmetic – Addition



A	B	A + B	
		Result	Carry
0	0	0	0
1	0	1	0
0	1	1	0
1	1	0	1



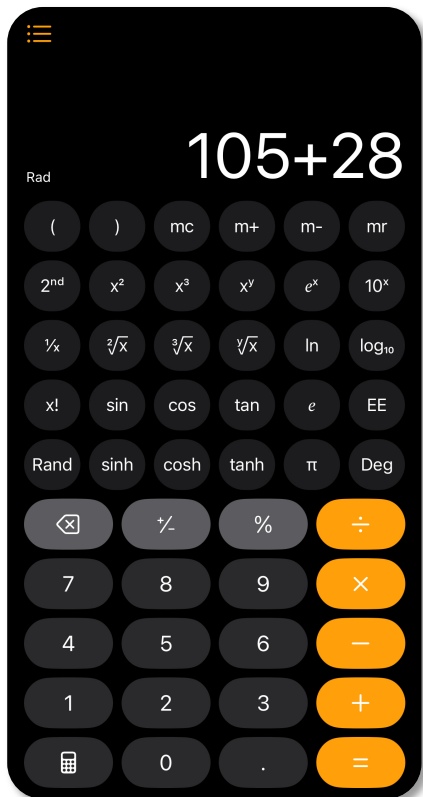
So, to compute **105 + 28** in *binary*, let's first convert 105 and 28 from *denary*...

$$\begin{array}{r} 1101001 \\ + 11100 \\ \hline 10000101 \end{array}$$

...and then we add bit-by-bit (remembering to carry).



Binary Arithmetic – Addition



So, to compute **105 + 28** in *binary*, let's first convert 105 and 28 from *denary*...

$$\begin{array}{r} 1101001 \\ + 11100 \\ \hline 10000101 = 133 \end{array}$$

...and then we add bit-by-bit (remembering to carry).



Binary Arithmetic – Addition

What should we do in this situation? What is `1 + 1 + 1`?

$$\begin{array}{r} 11 \\ + 11 \\ \hline 10 \end{array}$$

The diagram illustrates the binary addition of 11 and 11. The numbers are written in green. A red box highlights the carry '1' and the first digit '0' of the result '10'. The carry '1' is written in red below the first digit of the result.

`1 + 1 = 0 carry 1`, so adding another 1 gives us `1 carry 1`.



Binary Arithmetic – Addition

What should we do in this situation? What is `1 + 1 + 1`?

$$\begin{array}{r} 11 \\ + 11 \\ \hline 110 \end{array}$$

`1 + 1 = 0 carry 1`, so adding another 1 gives us `1 carry 1`.



Binary Arithmetic – Shifting

We will not go over binary multiplication in this module, but multiplying a number by **any power of two** is easy!

$$\begin{array}{r} 11 \\ \ll 1 \\ \hline 110 \end{array}$$

As we saw in this example, doubling a number is the same as just **shifting** all bits once to the left.

This is called a bit shift operation: **$11 \ll 1$** is a bit shift one space to the left, which is the same as multiplying by 2.



Binary Arithmetic – Shifting

Performing a *left* bit shift n many times will *multiply* the input by 2^n .

10011
<< 110
10011000000



Binary Arithmetic – Shifting

Performing a *right* bit shift n many times will *divide* the input by 2^n .

$$\begin{array}{r} 10011 \\ >> 10 \\ \hline 100 \end{array}$$

Note that the last two bits have now been completely lost, because we can currently only represent whole numbers. The right bit shift therefore performs an *integer* division, rather than a *real* division.



Binary Arithmetic – Overflow

Thus far, we have implicitly assumed that we have an **unlimited** number of bits available for each number – just like we do in real life when writing on paper or a whiteboard!

However, on a computer this is not the case:

- In a given type of computer, the size of integers is **fixed**
- 32 or 64 bits are popular choices these days



Binary Arithmetic – Overflow

The problem is, as we have seen, the result of binary arithmetic often requires more bits than the size of the inputs:

- Adding two n -bit numbers can yield a result of size $n+1$
- Left shifting can yield results much bigger than the input

$$\begin{array}{r} 1101001 \\ + 11100 \\ \hline 10000101 \end{array}$$

$$\begin{array}{r} 10011 \\ << 110 \\ \hline 10011000000 \end{array}$$



Binary Arithmetic – Overflow

Let's pretend we are working on a hypothetical computer that represents numbers as **7-bit** binary words.

Both of these operations give results that are over **7-bits**...

1101001

+ 11100

0000101

10011

<< 110

1000000

This is called **overflow**. When overflow occurs, an error could be raised, or the bits might simply **and unknowingly** be lost!



Binary Arithmetic – Overflow

Overflow can cause errors or, worse, wrong answers.

The below calculations say $105 + 28 = 5$ and $19 * 64 = 64$!

1101001

+ 11100

0000101

10011

<< 110

1000000



Binary Arithmetic – Overflow

In order to avoid overflow *when programming*, we should use appropriate data types.

For example, some programming languages have many different types for **integers** (whole numbers):

- **short** is the type of **16-bit** integers
- **int** is the type of **32-bit** integers
- **long** is the type of **64-bit** integers

However, Python's int type is of (effectively) unlimited size! **How?**

(Feel free to look into it and let me know if you find out how it works.)



Binary Arithmetic – Subtraction

The final arithmetic operation we would like to introduce is **subtraction**.

101100

- 110



It is rather tricky to perform subtraction on binary numbers...
...but we can cheat by simply *adding* a **negative** number instead,
because **$A - B = A + (-B)$** .

However, to do *that* we'll first have to learn how to represent negative numbers.



-1011 0111?

05

Representing Negative Numbers

Let's explore how we represent numbers **below zero**. By doing this, we will illuminate how to perform **subtraction** on binary numbers.



Representing Negative Numbers

Let's say that our computer represents numbers as single-byte (8-bit) binary words using the usual binary representation.

The **maximum** number that can be represented is **255**.

128	64	32	16	8	4	2	1
1	1	1	1	1	1	1	1



Representing Negative Numbers

Let's say that our computer represents numbers as single-byte (8-bit) binary words using the usual binary representation.

The **minimum** number that can be represented is **0**.

128	64	32	16	8	4	2	1
0	0	0	0	0	0	0	0



Representing Negative Numbers

The `int` datatype includes both positive and negative numbers, but the representation we have explored only represents positive numbers.

We will look at two methods for representing **negative numbers**:

Sign-and-magnitude

The MSB is replaced with a **sign bit**, which acts like a '-' sign



Two's complement

The **more common** method, which makes the MSB itself negative



Negative Numbers – Sign-and-magnitude

How do *we* represent negative numbers?

We just write ‘-’ before the number!

128	64	32	16	8	4	2	1

We would like to do something similar in binary... but how can we denote whether or not there is a minus sign before the number?

With yet another binary bit!



Negative Numbers – Sign-and-magnitude

In the sign-and-magnitude representation, the MSB (in this 8-bit case, 2^7) instead becomes a **sign** bit.

When this bit is set to 1, it denotes the presence of a '-' sign.

-	64	32	16	8	4	2	1
1	1	1	0	0	1	0	0

The remaining bits are the **magnitude** of the number: they represent the *absolute value* of the represented number in the usual way.

Which *denary* number is represented by the sign-and-magnitude binary number **11100100**?



Negative Numbers – Sign-and-magnitude

In the sign-and-magnitude representation, the MSB (in this 8-bit case, 2^7) instead becomes a **sign** bit.

When this bit is set to 1, it denotes the presence of a '-' sign.

–	64	32	16	8	4	2	1
1	1	1	0	0	1	0	0

100

The magnitude of the number is **100**.



Negative Numbers – Sign-and-magnitude

In the sign-and-magnitude representation, the MSB (in this 8-bit case, 2^7) instead becomes a **sign** bit.

When this bit is set to 1, it denotes the presence of a '-' sign.

–	64	32	16	8	4	2	1
1	1	1	0	0	1	0	0

– 100

The magnitude of the number is **100**.

The sign of the number is **negative**.



Negative Numbers – Sign-and-magnitude

In the sign-and-magnitude representation, the MSB (in this 8-bit case, 2^7) instead becomes a **sign** bit.

When this bit is set to 1, it denotes the presence of a '-' sign.

–	64	32	16	8	4	2	1
1	1	1	0	0	1	0	0

–100

The represented number is **-100**.



Negative Numbers – Sign-and-magnitude

In the sign-and-magnitude representation, the MSB (in this 8-bit case, 2^7) instead becomes a **sign** bit.

When this bit is set to 1, it denotes the presence of a '-' sign.

–	64	32	16	8	4	2	1
0	1	1	0	0	1	0	0

100

The represented number is **-100**.

If the sign bit had been 0, then the represented number would have been **100**.



Negative Numbers – Sign-and-magnitude

For an **8-bit** sign-and-magnitude binary word...

–	64	32	16	8	4	2	1
1	1	1	1	1	1	1	1

What is the *maximum* representable number? **127**

What is the *minimum* representable number? **-127**



Negative Numbers – Sign-and-magnitude

The sign-and-magnitude representation seems sensible, simple and reflects what humans do with denary. Why not use it all the time?

Mathematically, sign-and-magnitude isn't ideal: it doesn't let us define **subtraction**!

Take the following law of subtraction: **$X - X = 0$** .

$$\begin{array}{r} 81 \\ - 81 \\ \hline \end{array}$$



Negative Numbers – Sign-and-magnitude

The sign-and-magnitude representation seems sensible, simple and reflects what humans do with denary. Why not use it all the time?

Mathematically, sign-and-magnitude isn't ideal: it doesn't let us define **subtraction**!

Take the following law of subtraction: **$X - X = 0$** .

$$\begin{array}{r} 81 \\ + -81 \\ \hline \end{array}$$



Negative Numbers – Sign-and-magnitude

The sign-and-magnitude representation seems sensible, simple and reflects what humans do with denary. Why not use it all the time?

Mathematically, sign-and-magnitude isn't ideal: it doesn't let us define **subtraction**!

Take the following law of subtraction: **$X - X = 0$** .

$$\begin{array}{r} 01010001 \\ + 11010001 \\ \hline 100100010 \end{array}$$



Negative Numbers – Sign-and-magnitude

The sign-and-magnitude representation seems sensible, simple and reflects what humans do with denary. Why not use it all the time?

Mathematically, sign-and-magnitude isn't ideal: it doesn't let us define **subtraction**!

Take the following law of subtraction: **$X - X = 0$** .

$$\begin{array}{r} 01010001 \\ + 11010001 \\ \hline 00100010 \end{array}$$

Overflow is removed!



Negative Numbers – Sign-and-magnitude

The sign-and-magnitude representation seems sensible, simple and reflects what humans do with denary. Why not use it all the time?

Mathematically, sign-and-magnitude isn't ideal: it doesn't let us define **subtraction**!

Take the following law of subtraction: **$X - X = 0$** .

The law doesn't hold with this representation. To define subtraction, we need to look elsewhere...

$$\begin{array}{r} 81 \\ - 81 \\ \hline 34 \end{array}$$



Negative Numbers – Two's complement

In the two's complement representation, the MSB (in this 8-bit case, 2^7) represents the **negated form of the original MSB**.

When this bit is set to 1, it denotes the subtraction of the MSB.

-128	64	32	16	8	4	2	1
1	1	1	0	0	1	0	0

$$-128 + 64 + 32 + 4$$

Which *denary* number is represented by the two's complement binary number **11100100**?



Negative Numbers – Two's complement

In the two's complement representation, the MSB (in this 8-bit case, 2^7) represents the **negated form of the original MSB**.

When this bit is set to 1, it denotes the subtraction of the MSB.

-128	64	32	16	8	4	2	1
1	1	1	0	0	1	0	0

-28

The represented number is **-28**.



Negative Numbers – Two's complement

In the two's complement representation, the MSB (in this 8-bit case, 2^7) represents the **negated form of the original MSB**.

When this bit is set to 1, it denotes the subtraction of the MSB.

-128	64	32	16	8	4	2	1
0	1	1	0	0	1	0	0

100

The represented number is **-28**.

If the MSB had been 0, then the represented number would have been **100**.



Negative Numbers – Two's complement

For an **8-bit** two's complement binary word...

-128	64	32	16	8	4	2	1
1	0	0	0	0	0	0	0

What is the *maximum* representable number? **127**

What is the *minimum* representable number? **-128**

How is it possible that we can represent more numbers with two's complement? **Because there is only one representation of zero!**



Negative Numbers – Two's complement

Negating a number in sign-and-magnitude is easy: we just put a 1 in the MSB.

But two's complement is a bit less intuitive than this.

-128	64	32	16	8	4	2	1

So how do we convert from denary to two's complement binary?
Let's take the denary number **-81**.



Negative Numbers – Two's complement

Negating a number in sign-and-magnitude is easy: we just put a 1 in the MSB.

But two's complement is a bit less intuitive than this.

-128	64	32	16	8	4	2	1
0	1	0	1	0	0	0	1

So how do we convert from denary to two's complement binary?
Let's take the denary number **-81**.

1. Convert its magnitude **81** into binary.



Negative Numbers – Two's complement

Negating a number in sign-and-magnitude is easy: we just put a 1 in the MSB.

But two's complement is a bit less intuitive than this.

-128	64	32	16	8	4	2	1
1	0	1	0	1	1	1	0

So how do we convert from denary to two's complement binary?
Let's take the denary number **-81**.

1. Convert its magnitude **81** into binary.
2. Flip every bit.



Negative Numbers – Two's complement

Negating a number in sign-and-magnitude is easy: we just put a 1 in the MSB.

But two's complement is a bit less intuitive than this.

-128	64	32	16	8	4	2	1
1	0	1	0	1	1	1	1

So how do we convert from denary to two's complement binary?
Let's take the denary number **-81**.

1. Convert its magnitude **81** into binary.
2. Flip every bit.
3. Add **1** to the number, an ignoring any overflow.
4. Don't forget to check your answer!



Negative Numbers – Two's complement

Negating a number in sign-and-magnitude is easy: we just put a 1 in the MSB.

But two's complement is a bit less intuitive than this.

-128	64	32	16	8	4	2	1
1	0	1	0	1	1	1	1

So how do we convert from denary to two's complement binary?
Let's take the denary number **-81**.

1. Convert its magnitude **81** into binary.
2. Flip every bit.
3. Add **1** to the number, an ignoring any overflow.
4. Don't forget to check your answer! **$-128 + 32 + 8 + 4 + 2 + 1 = -81$**



Negative Numbers – Two's complement

The sign-and-magnitude representation seems sensible, simple and reflects what humans do with denary. Why not use it all the time?

Mathematically, sign-and-magnitude isn't ideal: it doesn't let us define **subtraction**!

Take the following law of subtraction: **$X - X = 0$** .

$$\begin{array}{r} 01010001 \\ + 10101111 \\ \hline 100000000 \end{array}$$



Negative Numbers – Two's complement

The sign-and-magnitude representation seems sensible, simple and reflects what humans do with denary. Why not use it all the time?

Mathematically, sign-and-magnitude isn't ideal: it doesn't let us define **subtraction**!

Take the following law of subtraction: **$X - X = 0$** .

$$\begin{array}{r} 01010001 \\ + 10101111 \\ \hline 00000000 \end{array}$$

Overflow is removed!



Negative Numbers – Two's complement

The sign-and-magnitude representation seems sensible, simple and reflects what humans do with denary. Why not use it all the time?

Mathematically, sign-and-magnitude isn't ideal: it doesn't let us define **subtraction**!

Take the following law of subtraction: **$X - X = 0$** .

The law holds with this representation. We now know how to perform subtraction on binary numbers!

$$\begin{array}{r} 81 \\ - 81 \\ \hline 0 \end{array}$$



Negative Numbers – Two's complement

Why *does* subtraction work for two's complement?

It's all to do with **overflow**!

255 + 1 in 8-bit binary arithmetic “looks like” 0, once we remove overflow.

255 is thus *behaving like* **-1**.

So, once we take out the MSB, we represent a negative number **-X** as the usual binary representation of **256 - X**.

$$\begin{array}{r} 11111111 \\ + 00000001 \\ \hline 00000000 \end{array}$$



Negative Numbers – Two's complement

Why *does* subtraction work for two's complement?

It's all to do with **overflow**!

2^n in n -bit binary arithmetic "looks like" 0.

$2^n - 1$ is thus *behaving like* -1.

So, once we take out the MSB, we represent a negative number $-X$ as the usual binary representation of $2^n - X$.

Let's
generalise!



Representing Negative Numbers

Let's review the three representations of integers for **8-bit** arithmetic:

Representation	Minimum	Maximum
Usual	0	255
Sign-and-magnitude	-127	127
Two's complement	-128	127



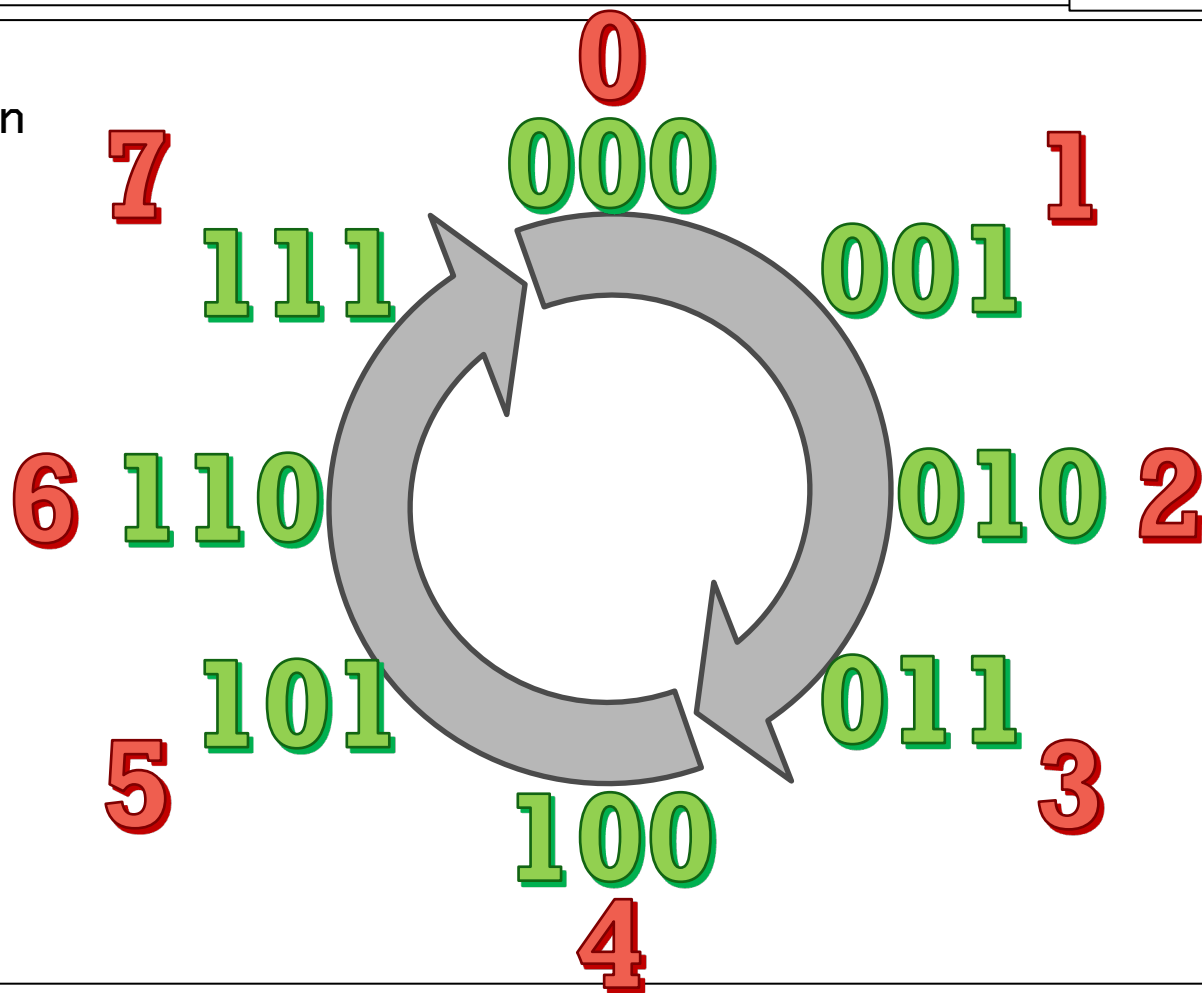
Representing Negative Numbers

Let's generalise this to **n-bit** arithmetic:

Representation	Minimum	Maximum
Usual	0	$2^n - 1$
Sign-and-magnitude	$-(2^{n-1} - 1)$	$2^{n-1} - 1$
Two's complement	-2^{n-1}	$2^{n-1} - 1$

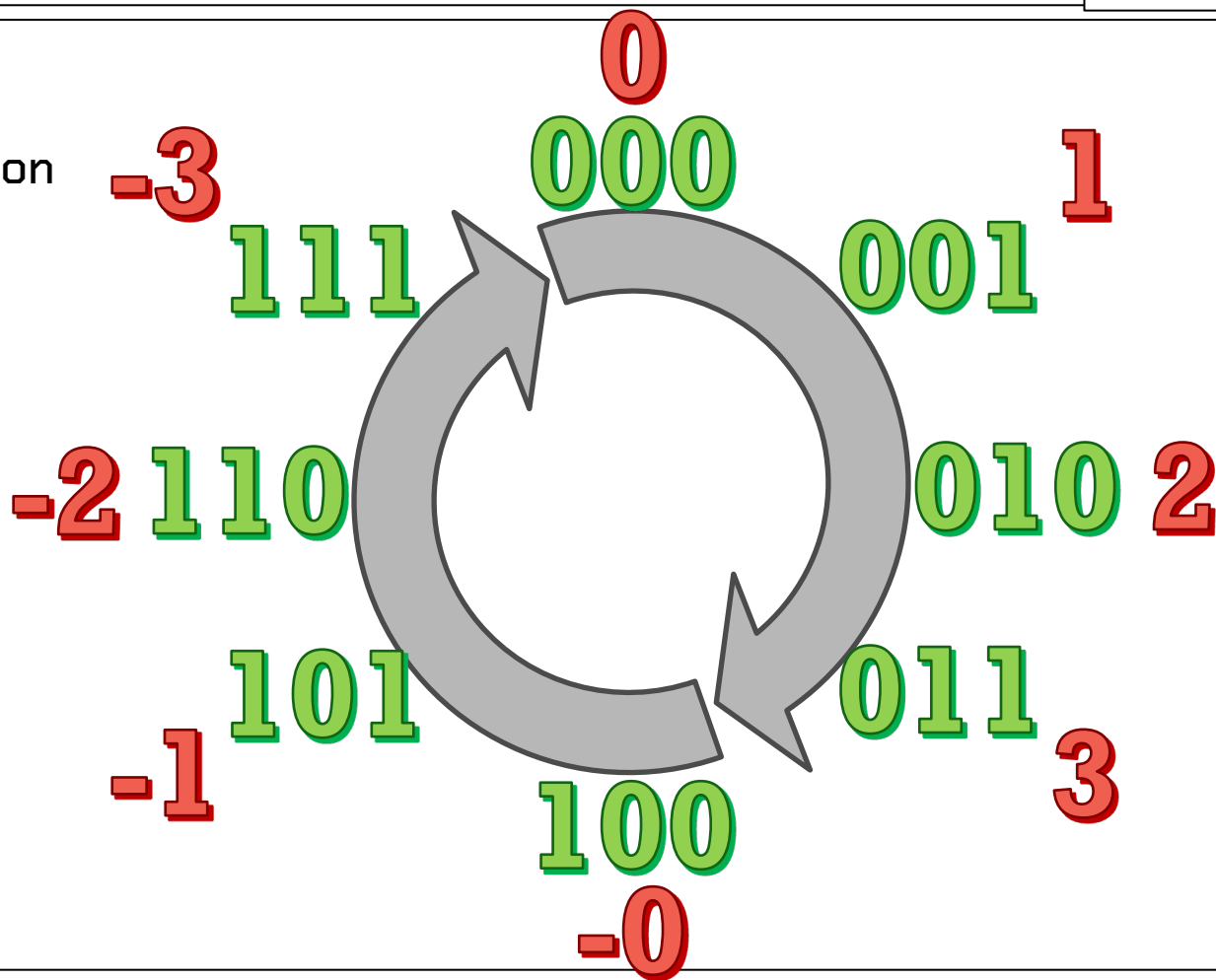


Usual
representation



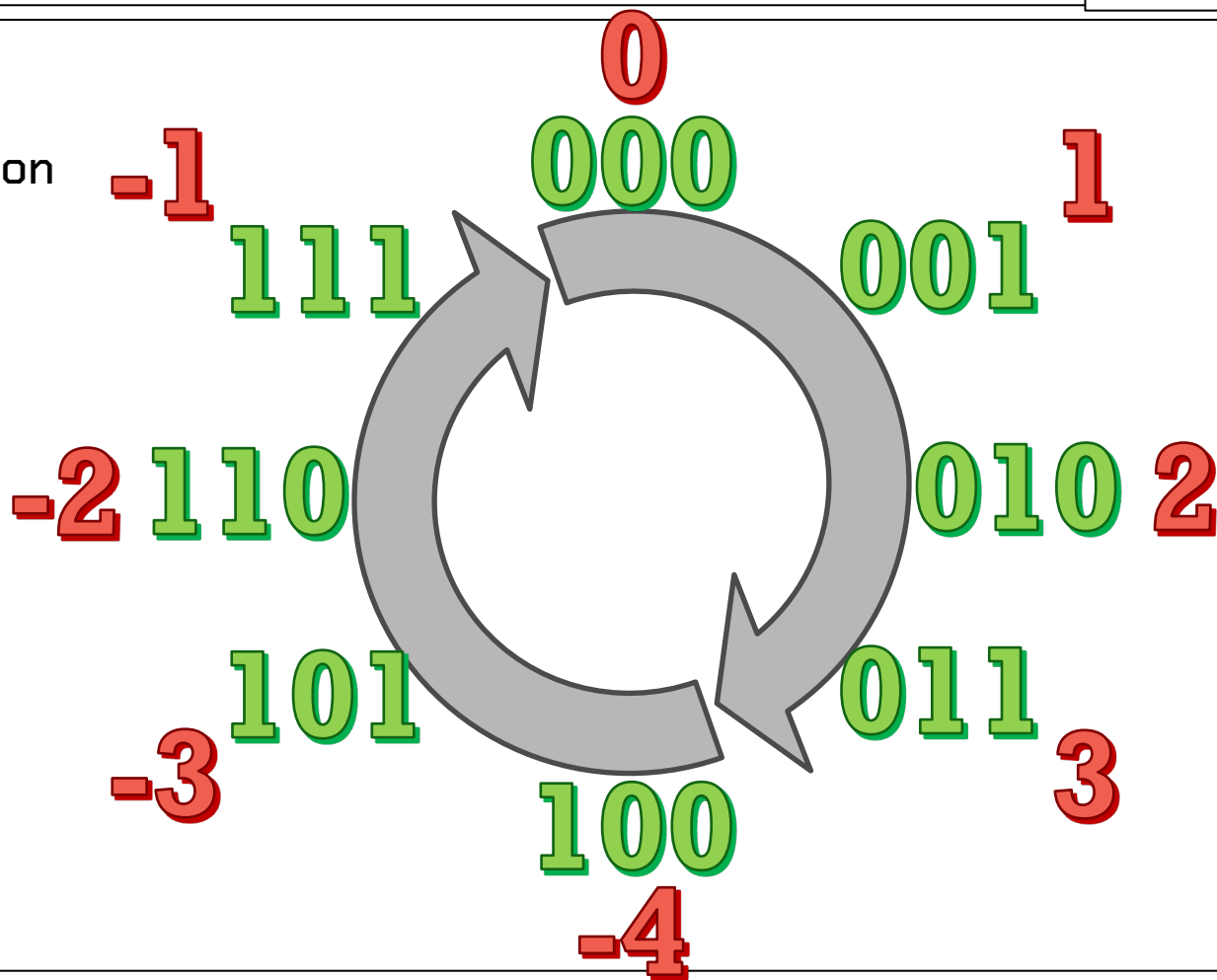


Sign-and-
magnitude
representation





Two's
complement
representation





$1.0 / 2 = 0.1?$

06

Representing Real Numbers

Finally, let's explore how we represent
numbers **with a decimal point**.



Representing Real Numbers

The **float** datatype includes both positive and negative *fractional* numbers, but the representations we have explored thus far have only represented *whole* numbers (also called **integers**).

We will look at two methods for representing **real numbers**:

Fixed point

An intuitive method, wherein a binary number has a **fixed binary point**



Floating point

The **standard** approach, wherein the binary point is not fixed, but **'floats'**



Real Numbers – Fixed point

How do *we* represent real numbers?

We separate the number into an *integer part* and a *fractional part* with a decimal point '.'!

2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0

We can do something similar in binary very easily.

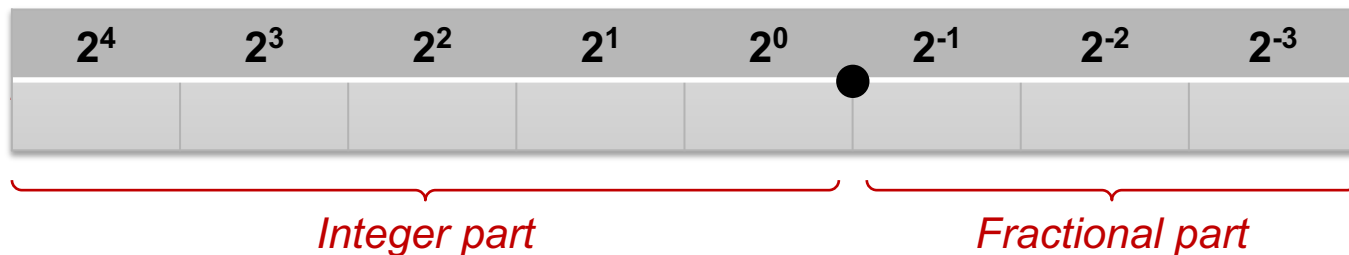
Let's say we have a fixed representation of 8-bits, we can choose to split those bits into (for example) a 5-bit Integer part and a 3-bit fractional part.



Real Numbers – Fixed point

The integer part is straightforward; but what should the **value** of the fractional part's bits be?

In *denary*, these would be 0.1, 0.01, 0.001...



...so in *binary*, they should be 0.5, 0.25, 0.125!



Real Numbers – Fixed point

Binary-to-denary works exactly the same with fixed point binary numbers.

16	8	4	2	1		0.5	0.25	0.125
1	0	1	1	1	●	0	1	1
<i>Integer part</i>						<i>Fractional part</i>		

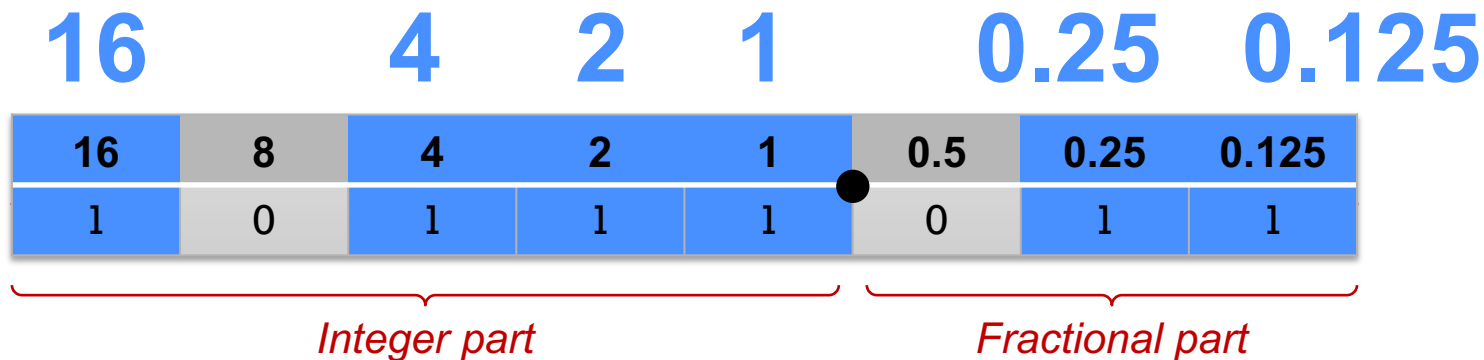
What denary number does **10111011** represent in our 8-bit (5-bit integer part, 3-bit fractional part) fixed point representation?

(Note that we would often write this as **10111.011** to make the binary point explicit.)



Real Numbers – Fixed point

Binary-to-denary works exactly the same with fixed point binary numbers.



What denary number does **10111011** represent in our 8-bit (5-bit integer part, 3-bit fractional part) fixed point representation?

(Note that we would often write this as **10111.011** to make the binary point explicit.)



Real Numbers – Fixed point

Binary-to-denary works exactly the same with fixed point binary numbers.

$$16 + 4 + 2 + 1 + 0.25 + 0.125$$

16	8	4	2	1	0.5	0.25	0.125
1	0	1	1	1	0	1	1

Integer part

Fractional part

What denary number does **10111011** represent in our 8-bit (5-bit integer part, 3-bit fractional part) fixed point representation?

(Note that we would often write this as **10111.011** to make the binary point explicit.)



Real Numbers – Fixed point

Binary-to-denary works exactly the same with fixed point binary numbers.

23.375

16	8	4	2	1	0.5	0.25	0.125
1	0	1	1	1	0	1	1

Integer part *Fractional part*

What denary number does **10111011** represent in our 8-bit (5-bit integer part, 3-bit fractional part) fixed point representation?

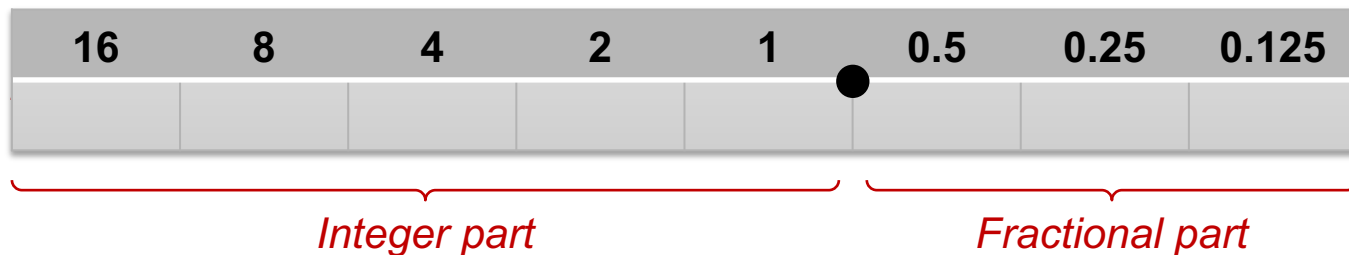
(Note that we would often write this as **10111.011** to make the binary point explicit.)



Real Numbers – Fixed point

The *left-to-right* **denary-to-binary** method also still works.

9.625

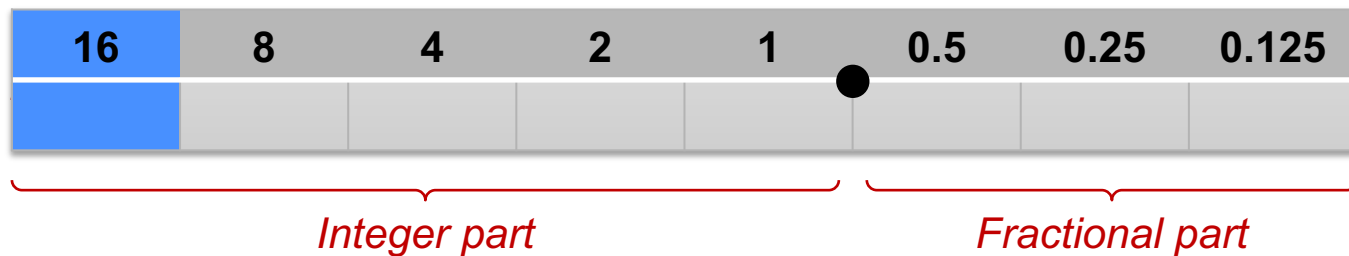




Real Numbers – Fixed point

The *left-to-right* **denary-to-binary** method also still works.

9.625

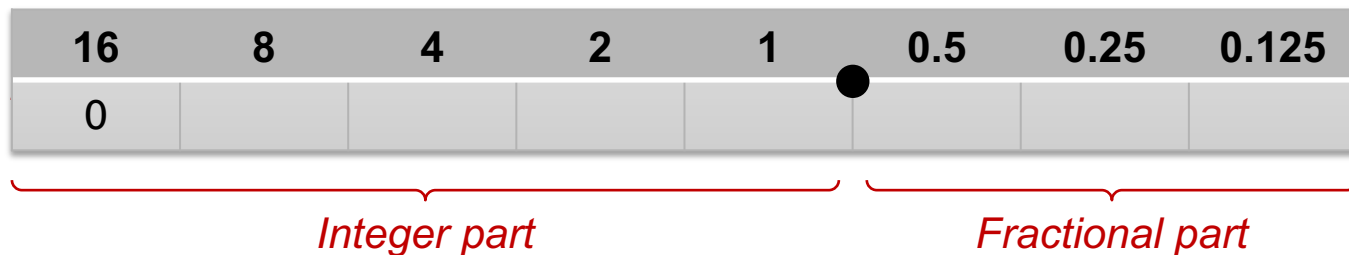




Real Numbers – Fixed point

The *left-to-right* **denary-to-binary** method also still works.

9.625

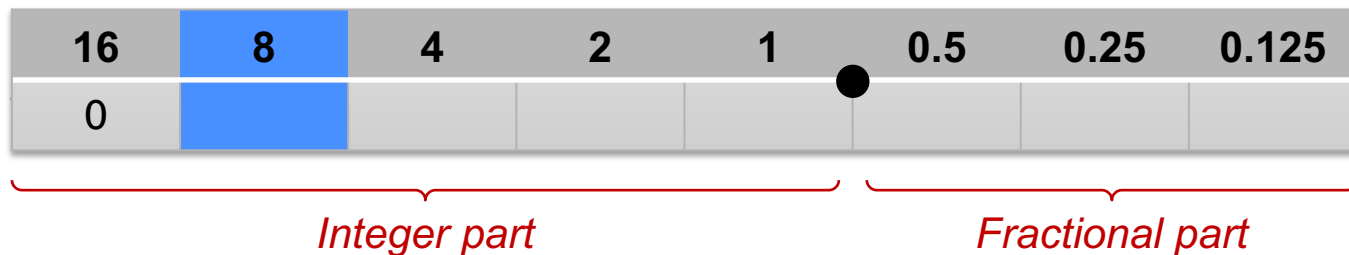




Real Numbers – Fixed point

The *left-to-right* **denary-to-binary** method also still works.

9.625

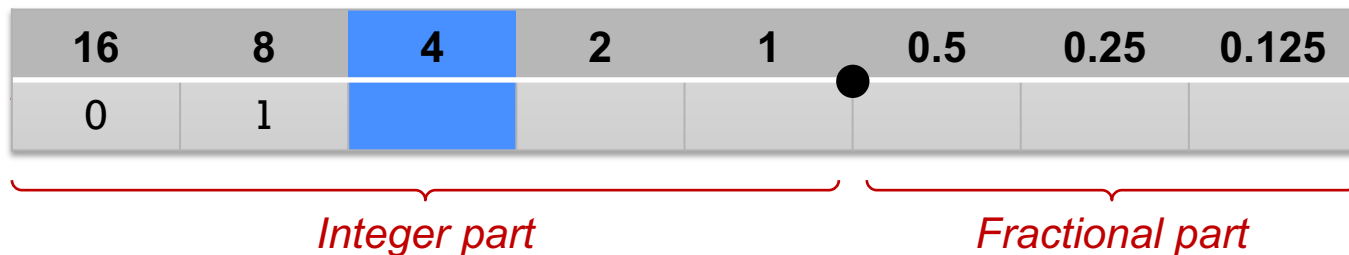




Real Numbers – Fixed point

The *left-to-right* **denary-to-binary** method also still works.

1.625

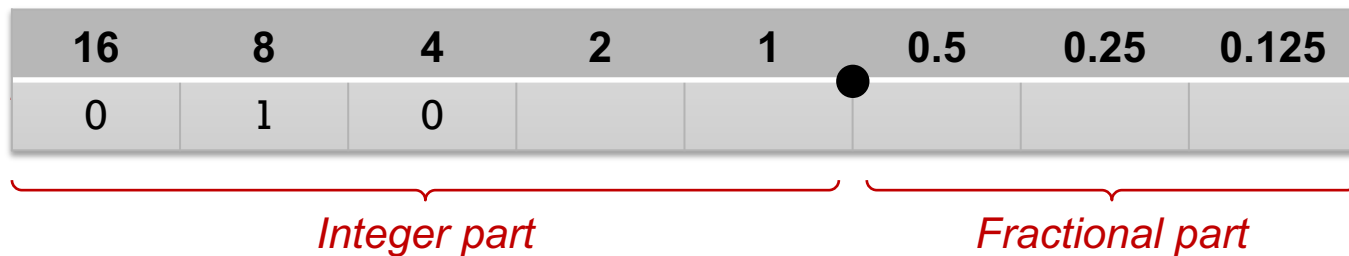




Real Numbers – Fixed point

The *left-to-right* **denary-to-binary** method also still works.

1.625

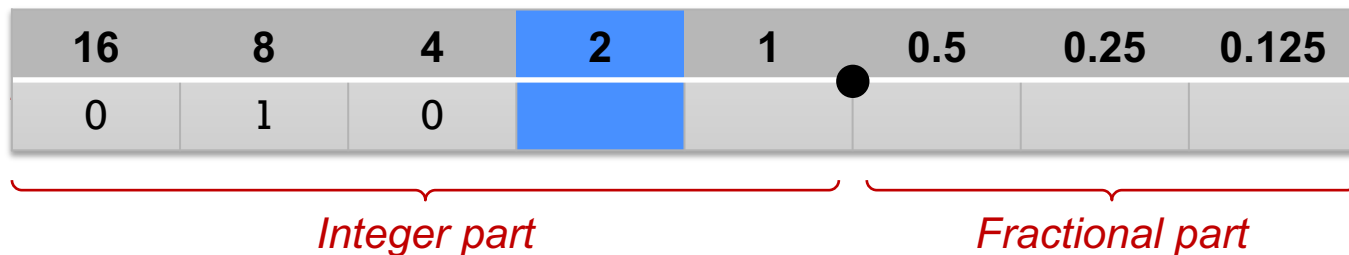




Real Numbers – Fixed point

The *left-to-right* **denary-to-binary** method also still works.

1.625

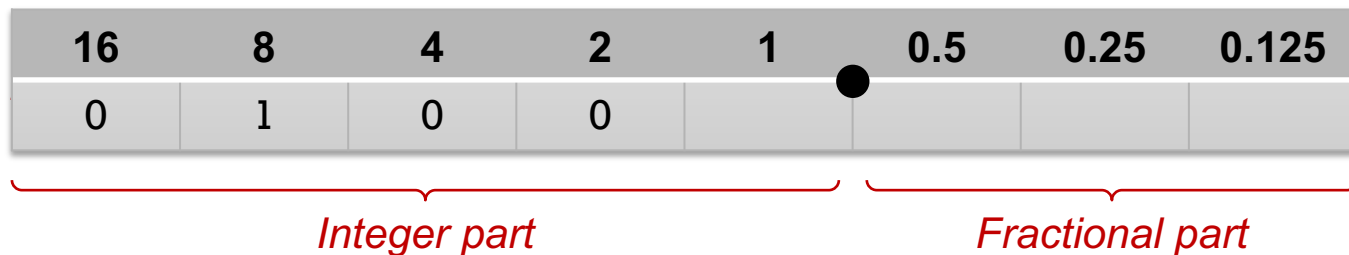




Real Numbers – Fixed point

The *left-to-right* **denary-to-binary** method also still works.

1.625

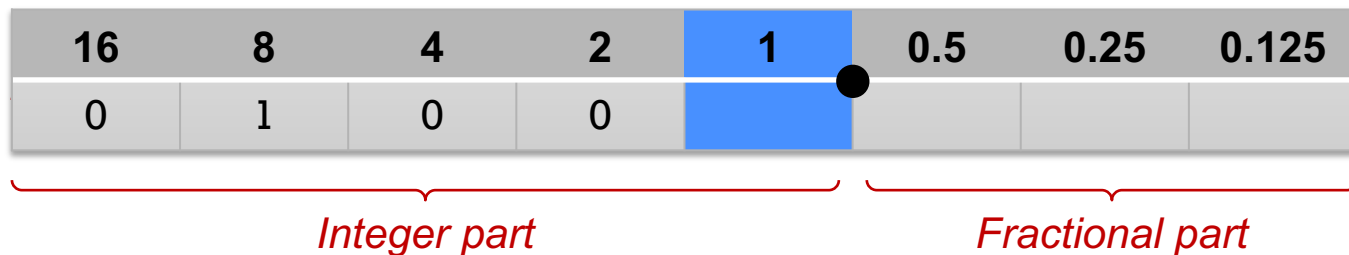




Real Numbers – Fixed point

The *left-to-right* **denary-to-binary** method also still works.

1.625





Real Numbers – Fixed point

The *left-to-right* **denary-to-binary** method also still works.

0.625

16	8	4	2	1		0.5	0.25	0.125
0	1	0	0	1	●			

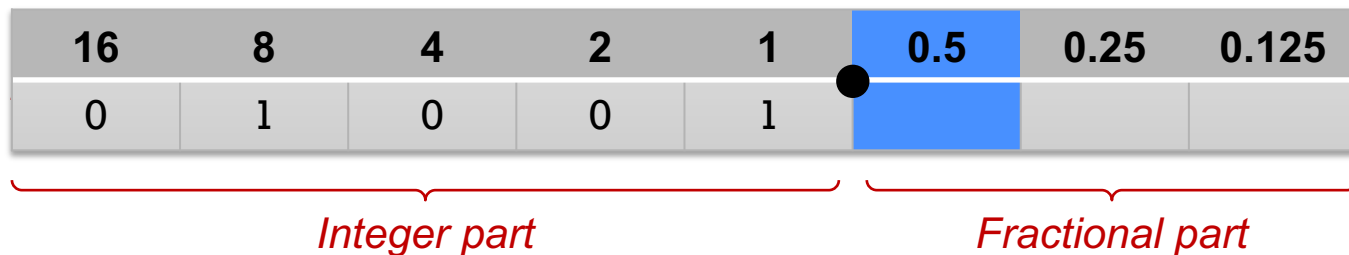
Integer part *Fractional part*



Real Numbers – Fixed point

The *left-to-right* **denary-to-binary** method also still works.

0.625





Real Numbers – Fixed point

The *left-to-right* **denary-to-binary** method also still works.

0.125

16	8	4	2	1		0.5	0.25	0.125
0	1	0	0	1	●	1		

Integer part *Fractional part*



Real Numbers – Fixed point

The *left-to-right* **denary-to-binary** method also still works.

0.125

16	8	4	2	1		0.5	0.25	0.125
0	1	0	0	1	•	1		

Integer part *Fractional part*



Real Numbers – Fixed point

The *left-to-right* **denary-to-binary** method also still works.

0.125

16	8	4	2	1		0.5	0.25	0.125
0	1	0	0	1	●	1	0	

Integer part *Fractional part*



Real Numbers – Fixed point

The *left-to-right* **denary-to-binary** method also still works.

0.125

16	8	4	2	1		0.5	0.25	0.125
0	1	0	0	1	●	1	0	
Integer part						Fractional part		



Real Numbers – Fixed point

The *left-to-right* **denary-to-binary** method also still works.

So, 9.625 is **01001.101** (or **01001101**) in this fixed point representation.

0

16	8	4	2	1		0.5	0.25	0.125
0	1	0	0	1	●	1	0	1

Integer part

Fractional part

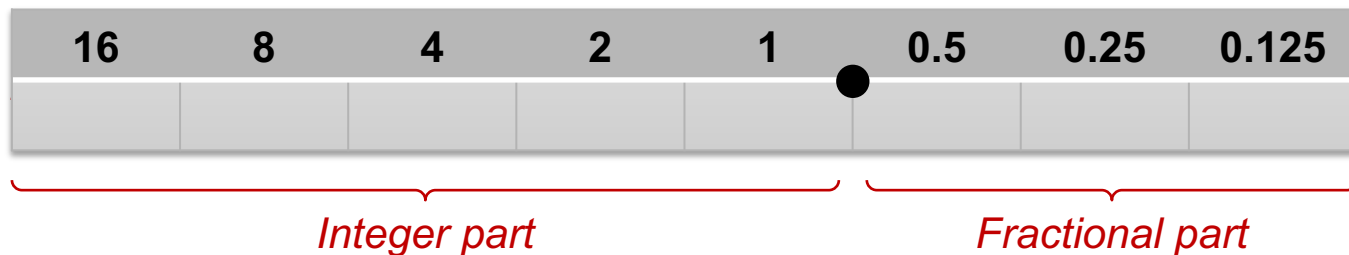


Real Numbers – Fixed point

The *right-to-left* **denary-to-binary** method is a bit different.

First, we compute the integer part of the number exactly the same as before (divide by 2 and keep the remainder).

9



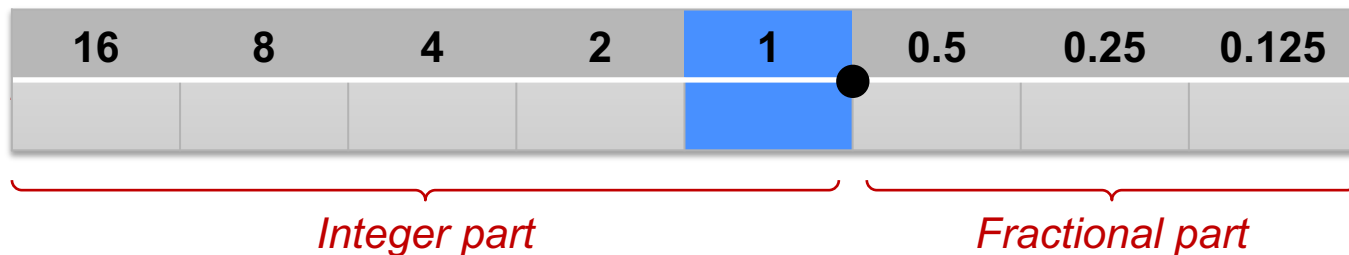


Real Numbers – Fixed point

The *right-to-left* **denary-to-binary** method is a bit different.

First, we compute the integer part of the number exactly the same as before (divide by 2 and keep the remainder).

4 rem 1



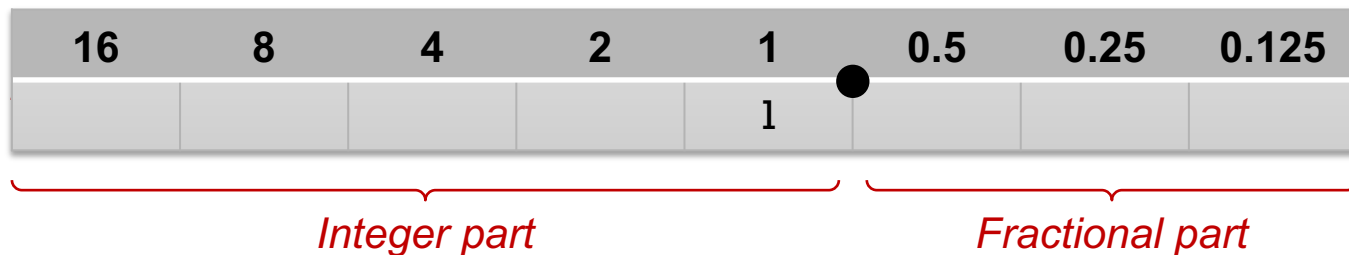


Real Numbers – Fixed point

The *right-to-left* **denary-to-binary** method is a bit different.

First, we compute the integer part of the number exactly the same as before (divide by 2 and keep the remainder).

4



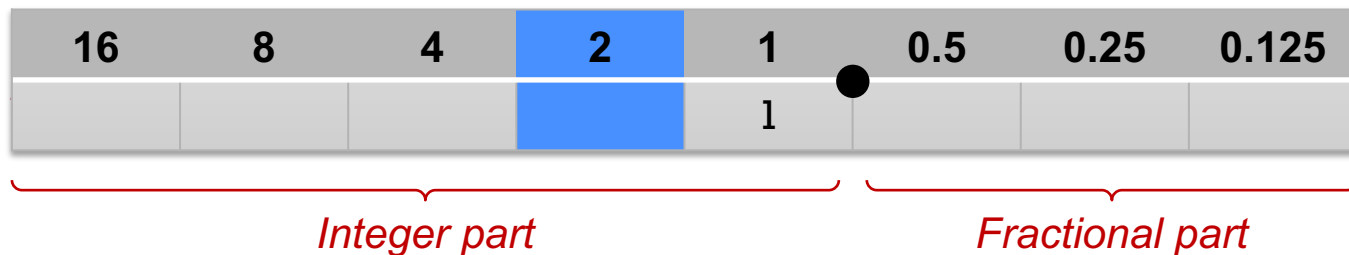


Real Numbers – Fixed point

The *right-to-left* **denary-to-binary** method is a bit different.

First, we compute the integer part of the number exactly the same as before (divide by 2 and keep the remainder).

2 rem 0





Real Numbers – Fixed point

The *right-to-left* **denary-to-binary** method is a bit different.

First, we compute the integer part of the number exactly the same as before (divide by 2 and keep the remainder).

2

16	8	4	2	1		0.5	0.25	0.125
			0	1	●			
Integer part						Fractional part		

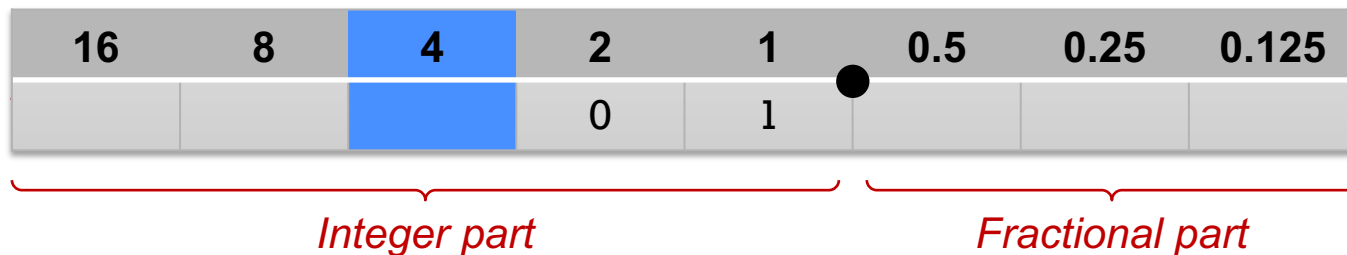


Real Numbers – Fixed point

The *right-to-left* **denary-to-binary** method is a bit different.

First, we compute the integer part of the number exactly the same as before (divide by 2 and keep the remainder).

1 rem 0





Real Numbers – Fixed point

The *right-to-left* **denary-to-binary** method is a bit different.

First, we compute the integer part of the number exactly the same as before (divide by 2 and keep the remainder).

1

16	8	4	2	1	0.5	0.25	0.125
		0	0	1			
Integer part					Fractional part		



Real Numbers – Fixed point

The *right-to-left* **denary-to-binary** method is a bit different.

First, we compute the integer part of the number exactly the same as before (divide by 2 and keep the remainder).

0 rem 1

16	8	4	2	1	0.5	0.25	0.125
		0	0	1			
Integer part					Fractional part		



Real Numbers – Fixed point

The *right-to-left* **denary-to-binary** method is a bit different.

First, we compute the integer part of the number exactly the same as before (divide by 2 and keep the remainder).

0

16	8	4	2	1	0.5	0.25	0.125
	1	0	0	1			

Integer part *Fractional part*



Real Numbers – Fixed point

The *right-to-left* **denary-to-binary** method is a bit different.

First, we compute the integer part of the number exactly the same as before (divide by 2 and keep the remainder).

16	8	4	2	1	0.5	0.25	0.125
0	1	0	0	1			

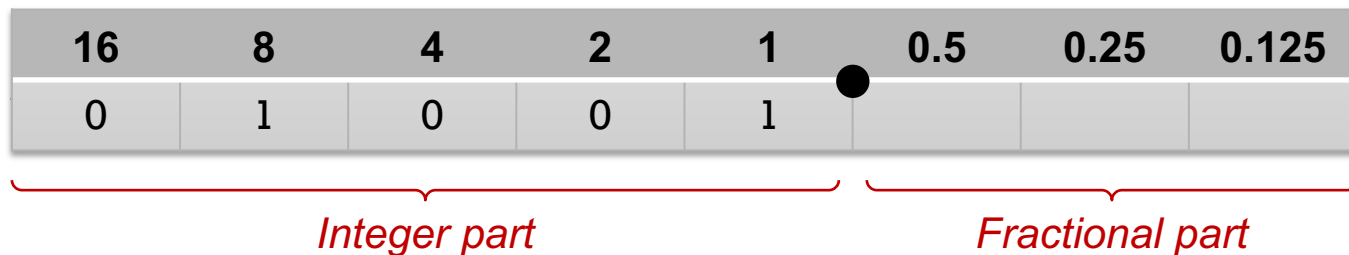
Integer part *Fractional part*



Real Numbers – Fixed point

For the fractional part, we do the opposite: we go *left-to-right* and **multiply by 2** rather than **divide by 2**.

0.625



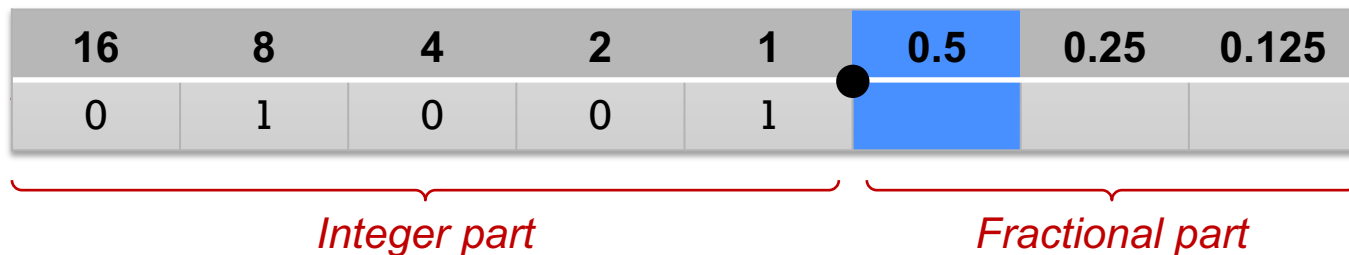
Following each multiplication, we place the integer part of the number in the next bit and keep the fractional part.

0.625



Real Numbers – Fixed point

For the fractional part, we do the opposite: we go *left-to-right* and **multiply by 2** rather than **divide by 2**.



Following each multiplication, we place the integer part of the number in the next bit and keep the fractional part.

0.625 * 2 = 1.25; write **1** and keep **0.25**



Real Numbers – Fixed point

For the fractional part, we do the opposite: we go *left-to-right* and **multiply by 2** rather than **divide by 2**.

16	8	4	2	1	0.5	0.25	0.125
0	1	0	0	1	1		

Integer part *Fractional part*

Following each multiplication, we place the integer part of the number in the next bit and keep the fractional part.

0.25



Real Numbers – Fixed point

For the fractional part, we do the opposite: we go *left-to-right* and **multiply by 2** rather than **divide by 2**.

16	8	4	2	1	0.5	0.25	0.125
0	1	0	0	1	1		

Integer part *Fractional part*

Following each multiplication, we place the integer part of the number in the next bit and keep the fractional part.

0.25 * 2 = 0.5; write **0** and keep **0.5**



Real Numbers – Fixed point

For the fractional part, we do the opposite: we go *left-to-right* and **multiply by 2** rather than **divide by 2**.

16	8	4	2	1	●	0.5	0.25	0.125
0	1	0	0	1		1	0	

Integer part *Fractional part*

Following each multiplication, we place the integer part of the number in the next bit and keep the fractional part.

0.5



Real Numbers – Fixed point

For the fractional part, we do the opposite: we go *left-to-right* and **multiply by 2** rather than **divide by 2**.

16	8	4	2	1	0.5	0.25	0.125
0	1	0	0	1	1	0	

Integer part *Fractional part*

Following each multiplication, we place the integer part of the number in the next bit and keep the fractional part.

0.5 * 2 = 1.0; write **1** and keep **0**



Real Numbers – Fixed point

For the fractional part, we do the opposite: we go *left-to-right* and **multiply by 2** rather than **divide by 2**.

16	8	4	2	1		0.5	0.25	0.125
0	1	0	0	1	●	1	0	1

Integer part *Fractional part*

Following each multiplication, we place the integer part of the number in the next bit and keep the fractional part.

0 reached!



Real Numbers – Fixed point

Another way to understand fixed point binary numbers is as scaled binary integers.

For example, let's use a 7-bit fixed point representation (4-bit integer, 3-bit fractional).

The number **3.5** can be represented as **0011100**.

8	4	2	1	0.5	0.25	0.125
0	0	1	1	1	0	0

In the usual positive integer representation, this would be the number **28**...

64	32	16	8	4	2	1
0	0	1	1	1	0	0

...and **$28 = 3.5 * 2^3$** .



Real Numbers – Fixed point

So an easy way to convert between an **n-bit** binary number **X** and a fixed point (with **i-bit** fractional part) binary number **B** is:

$$X = B * 2^i \quad (\text{or } X = B \ll i)$$

The number **3.5** can be represented as **0011100**.

In the usual positive integer representation, this would be the number **28**...

...and **28 = 3.5 * 2³**.

8	4	2	1	0.5	0.25	0.125
0	0	1	1	1	0	0

64	32	16	8	4	2	1
0	0	1	1	1	0	0

Left shift by 3



Of course, we can also represent negative real numbers.

How would you represent -59.25 as a 10-bit *sign-and-magnitude* fixed point binary number with a 2-bit fractional part?

A horizontal logarithmic scale with a gray background. The scale is marked with the values 64, 32, 16, 8, 4, 2, 1, 0.5, and 0.25 from left to right. A black dot is placed on the scale at the position corresponding to the value 1.



Real Numbers – Fixed point

Of course, we can also represent negative real numbers.

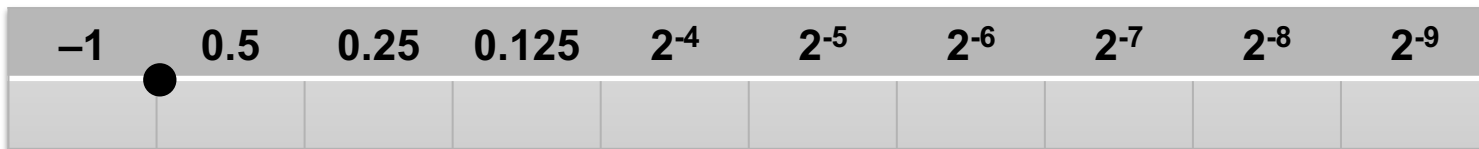
How would you represent -59.25 as a 10-bit *sign-and-magnitude* fixed point binary number with a 2-bit fractional part?

–	64	32	16	8	4	2	1	0.5	0.25
1	0	1	1	1	0	1	1	0	1



Of course, we can also represent negative real numbers.

How would you represent -0.533203125 as a 10-bit *two's complement* fixed point binary number with a 9-bit fractional part?





Real Numbers – Fixed point

Of course, we can also represent negative real numbers.

How would you represent -0.533203125 as a 10-bit *two's complement* fixed point binary number with a 9-bit fractional part?

-1	0.5	0.25	0.125	2^{-4}	2^{-5}	2^{-6}	2^{-7}	2^{-8}	2^{-9}
1	0	1	1	1	0	1	1	1	1

There are many ways of doing this. Here's how I worked this out:

- $-0.533203125 * 2^9 = -273$



Real Numbers – Fixed point

Of course, we can also represent negative real numbers.

How would you represent -0.533203125 as a 10-bit *two's complement* fixed point binary number with a 9-bit fractional part?

512	256	128	64	32	16	8	4	2	1
0	1	0	0	0	1	0	0	0	1

There are many ways of doing this. Here's how I worked this out:

- $-0.533203125 * 2^9 = -273$
- 273 in standard 10-bit binary = 0100010001 (using left-to-right method)



Real Numbers – Fixed point

Of course, we can also represent negative real numbers.

How would you represent -0.533203125 as a 10-bit *two's complement* fixed point binary number with a 9-bit fractional part?

-512	256	128	64	32	16	8	4	2	1
1	0	1	1	1	0	1	1	1	1

There are many ways of doing this. Here's how I worked this out:

- $-0.533203125 * 2^9 = -273$
- 273 in standard 10-bit binary = 0100010001 (using left-to-right method)
- Flip all bits and add 1 to convert to two's complement



Real Numbers – Fixed point

Of course, we can also represent negative real numbers.

How would you represent -0.533203125 as a 10-bit *two's complement* fixed point binary number with a 9-bit fractional part?

-1	0.5	0.25	0.125	2^{-4}	2^{-5}	2^{-6}	2^{-7}	2^{-8}	2^{-9}
1	0	1	1	1	0	1	1	1	1

There are many ways of doing this. Here's how I worked this out:

- $-0.533203125 * 2^9 = -273$
- 273 in standard 10-bit binary = 0100010001 (using left-to-right method)
- Flip all bits and add 1 to convert to two's complement



Real Numbers – Fixed point

Fixed point isn't perfect for representing real numbers.

Let's try to represent 0.03125 with a 5-bit fixed point representation with a 4-bit fractional part.

1	0.5	0.25	0.125	0.0625
0	0	0	0	0

So, in this representation, **0.03125 = 0!**



Real Numbers – Fixed point

Fixed point isn't perfect for representing real numbers.

Let's try to represent 0.03125 with a 5-bit fixed point representation with a 4-bit fractional part.

0.5	0.25	0.125	0.0625	0.03125
0	0	0	0	1

So, in this representation, **0.03125 = 0!**

If we had chosen a 5-bit fractional part, we could have represented it...



Real Numbers – Fixed point

Fixed point isn't perfect for representing real numbers.

Let's try to represent 0.03125 with a 5-bit fixed point representation with a 4-bit fractional part.

1	0.5	0.25	0.125	0.0625
1	1	0	0	0

So, in this representation, **0.03125 = 0!**

If we had chosen a 5-bit fractional part, we could have represented it...

...but *then* we couldn't have represented 1.5, for example.



Real Numbers – Fixed point

If we have access to a given number of bits for our numbers (in this case **5**), it would be helpful to be able to *change* how large or small the fractional part is **according to demand**.

1	0.5	0.25	0.125	0.0625
0	0	0	0	0

0.5	0.25	0.125	0.0625	0.03125
0	0	0	0	1

This is called a *floating point* representation.



Real Numbers – Floating point

A **floating point** representation is like **scientific notation**, but in *binary*.

Scientific notation is when a *denary* (and usually fractional) number is represented in the form:

$$m * 10^e$$

m is a *denary* (and usually fractional) number

e is a *denary* integer

For example:

- $1025.6 = 1.0256 * 10^3$
- $0.086 = 8.6 * 10^{-2}$
- $1.099 = 1.099 * 10^0$



Real Numbers – Floating point

We could follow this convention in exactly the same way in binary!

A *binary* (usually fractional) number will be written in the form:

$$m * 2^e$$

m is a *binary* (and usually fractional) number
 e is a *binary* integer

For example:

- $1011.1 = 1.0111 * 2^3$
- $0.011 = 1.1 * 2^{-2}$
- $1.011 = 1.011 * 2^0$



Real Numbers – Floating point

So, the idea of floating point is to represent binary numbers as two numbers, m and e , such that $n = m * 2^e$.

How can we store this in a (let's say 16-bit) computer?



The representation can be split into two parts: the part that represents m and the part that represents e .

In general, the *mantissa* m needs to have a higher number of bits than the *exponent* e , so let's have a 12-bit mantissa and a 4-bit exponent.



Real Numbers – Floating point

There are now three problems we need to deal with:

1. How should we represent the mantissa?
2. How should we represent negative numbers?
3. How should we represent the exponent?





Real Numbers – Floating point

In our examples of both *denary* and *binary* scientific notation, m was always **normalised**: this means it always had exactly **one** digit/bit before the decimal/binary point.



So, it would seem to make sense to represent m as a fixed point number, with only a 1-bit integer part...

However, this is not an efficient use of space. Why?



Real Numbers – Floating point

For binary numbers, for every number other than zero, the normalised scientific notation of the number will start with **1.**



We could simply remember that numbers should start with **1.** and effectively *hide* this extra bit.

(This does mean we now can't represent zero, but we will fix this later.)



Real Numbers – Floating point

1. How should we represent the mantissa?
2. How should we represent negative numbers?
3. How should we represent the exponent?
4. How should we represent zero?





Real Numbers – Floating point

Floating point numbers represent negative numbers by using a **sign bit**.



This sign bit gives the sign of the mantissa (and therefore the overall represented number), *not the exponent*.

In general, we can assume it works as it did in **sign-and-magnitude** representation: **0** for positive numbers, **1** for negative numbers.



Real Numbers – Floating point

1. How should we represent the mantissa?
2. How should we represent negative numbers?
3. How should we represent the exponent?
4. How should we represent zero?





Real Numbers – Floating point

The exponent needs to be able to take on both positive and negative values. So, presumably, it either needs its own **sign bit** or to be stored in **two's complement**.



However, both have their issues. In particular, they both cause binary numbers to be *ordered incorrectly* relative to the ordering on the representing bit-strings.

This makes comparing numbers in the computer *much* more inefficient!



Real Numbers – Floating point

In order to make sure floating point numbers could be compared quickly in the computer, two things need to be done.

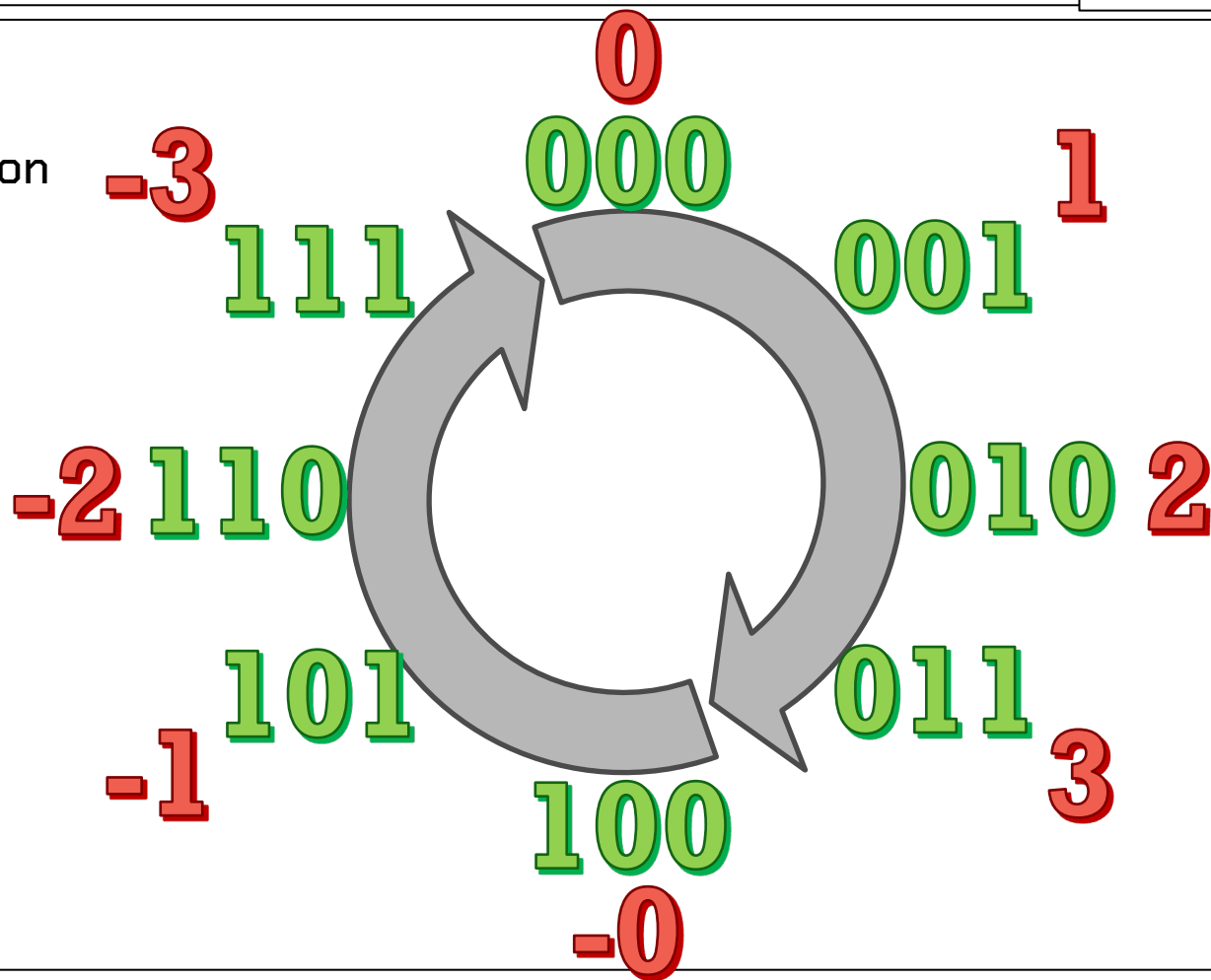
First, the **exponent** needs to come after the **sign bit** and before **m**.



Secondly, as introduced in 1954 with the IBM 704, the exponent needs to be stored with an **offset**.

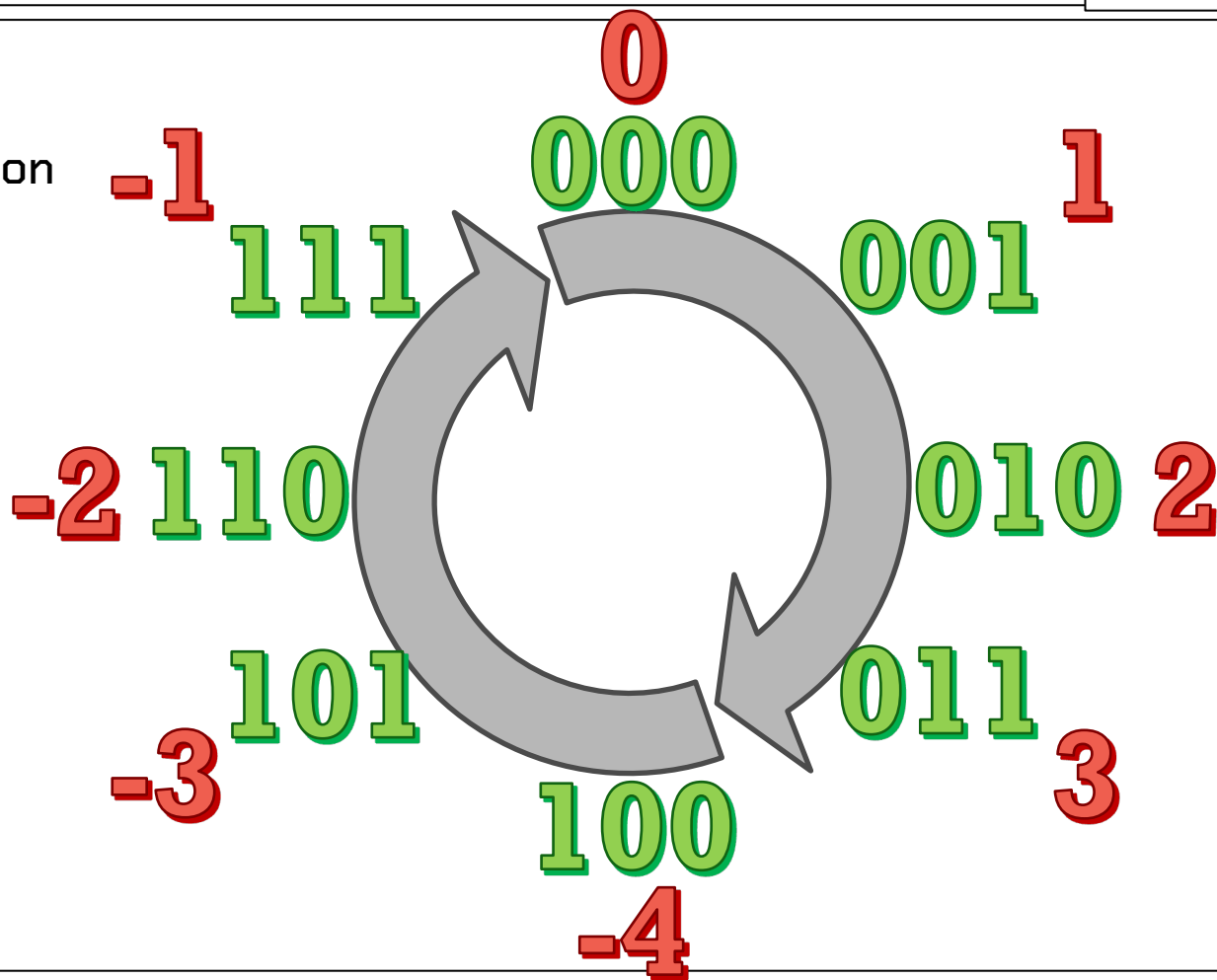


Sign-and-
magnitude
representation



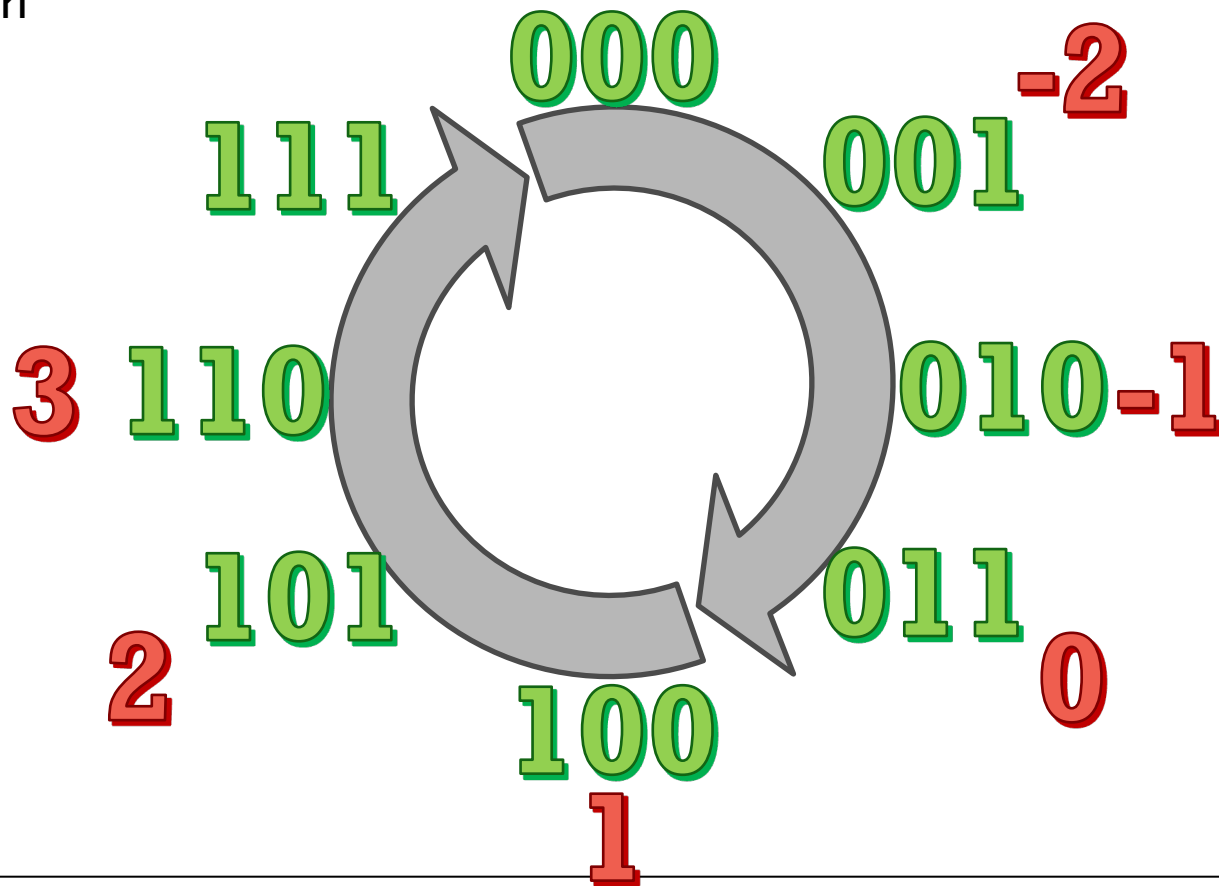


Two's
complement
representation





Offset/biased
representation
when $b = 3$





Real Numbers – Floating point

An **offset exponent**, sometimes called a **biased exponent**, is used instead of two's complement so that the representations of negative numbers are ordered *earlier* than the representations of positive numbers.



Given a bias b , the true exponent e can be computed from the offset exponent e^* by calculating $e = e^* - b$ (thus also $e^* = e + b$).

Additionally, as shown in the previous picture, the **all 0s** and **all 1s** values are not used – they are reserved for special values.



Real Numbers – Floating point

1. How should we represent the mantissa?
2. How should we represent negative numbers?
3. How should we represent the exponent?
3. How should we represent zero?

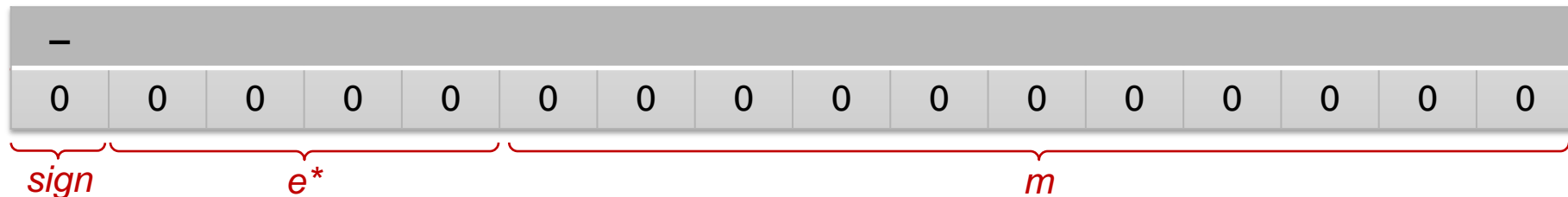




Real Numbers – Floating point

Because of the hidden bit, **zero** cannot be represented.

However, we reserved the special biased exponent value of **0000** for special values.



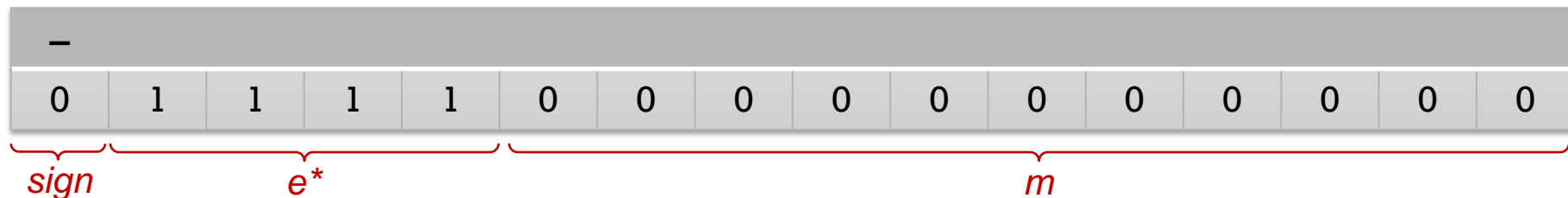
Zero is one of these special values: it is represented by a **0** sign bit, **0** biased exponent and **0** mantissa.



Real Numbers – Floating point

Other special values include $+\infty$, $-\infty$ and NaN.

Positive and negative infinity are the values which occur when floating point numbers overflow or underflow (respectively).



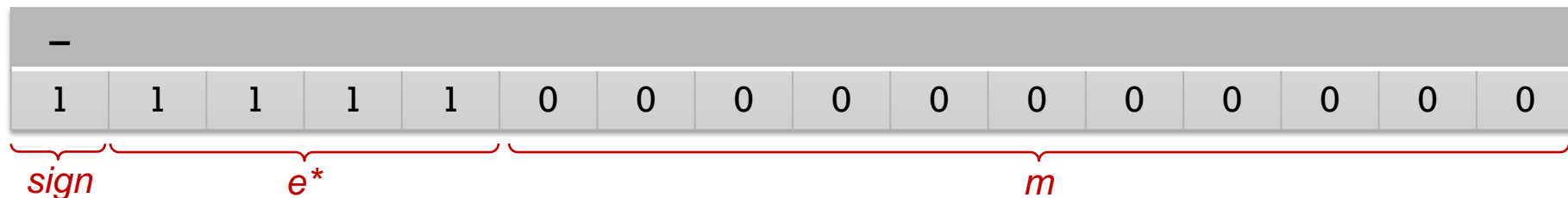
$+\infty$ is represented as a 0 sign bit, all 1s biased exponent and 0 mantissa.



Real Numbers – Floating point

Other special values include $+\infty$, $-\infty$ and NaN.

Positive and negative infinity are the values which occur when floating point numbers overflow or underflow (respectively).



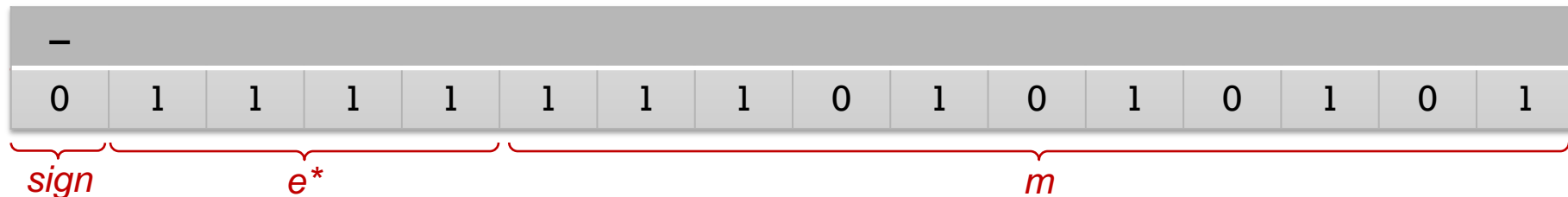
$-\infty$ is represented as a **1** sign bit, **all 1s** biased exponent and **0** mantissa.



Real Numbers – Floating point

Other special values include $+\infty$, $-\infty$ and NaN.

NaN means “not a number”, which occurs when we do strange things, like take the square root of a negative number.



NaN is represented as any sign bit, all 1s biased exponent and any mantissa except 0.



Real Numbers – Floating point

The standard for floating point arithmetic is *IEEE 754*. It consists of the features we introduced: a sign bit, a biased exponent, a mantissa with a hidden bit and special values such as $+\infty$, $-\infty$ and NaN.



IEEE 754 usually has one of the following precisions:

- Single precision (32-bit numbers with 8-bit exponent)
- Double precision (64-bit numbers with 11-bit exponent)



Let's do some examples with our hypothetical 16-bit (4-bit exponent) IEEE 754 representation, with an exponent bias of $b = 7$.

—	8	4	2	1	1024	512	256	128	64	32	16	8	4	2	1
1	0	1	1	0	0	1	1	0	0	1	1	0	0	0	0
<i>sign</i>		<i>e*</i>				<i>m</i>									



Real Numbers – Floating point

Let's do some examples with our hypothetical 16-bit (4-bit exponent) IEEE 754 representation, with an exponent bias of $b = 7$.

Which number is represented by the following?

–	8	4	2	1	1024	512	256	128	64	32	16	8	4	2	1
1	0	1	1	0	0	1	1	0	0	1	1	0	0	0	0
<i>sign</i>		<i>e*</i>				<i>m</i>									

The sign bit is **1**,



Real Numbers – Floating point

Let's do some examples with our hypothetical 16-bit (4-bit exponent) IEEE 754 representation, with an exponent bias of $b = 7$.

Which number is represented by the following?

–	8	4	2	1	1024	512	256	128	64	32	16	8	4	2	1
1	0	1	1	0	0	1	1	0	0	1	1	0	0	0	0
<i>sign</i>	<i>e*</i>				<i>m</i>										

The sign bit is **1**, so it's a negative number.



Real Numbers – Floating point

Let's do some examples with our hypothetical 16-bit (4-bit exponent) IEEE 754 representation, with an exponent bias of $b = 7$.

Which number is represented by the following?

–	8	4	2	1	1024	512	256	128	64	32	16	8	4	2	1
1	0	1	1	0	0	1	1	0	0	1	1	0	0	0	0
<i>sign</i>		<i>e*</i>				<i>m</i>									

The sign bit is **1**, so it's a negative number.
The biased exponent is **6**,



Real Numbers – Floating point

Let's do some examples with our hypothetical 16-bit (4-bit exponent) IEEE 754 representation, with an exponent bias of $b = 7$.

Which number is represented by the following?

–	8	4	2	1	1024	512	256	128	64	32	16	8	4	2	1
1	0	1	1	0	0	1	1	0	0	1	1	0	0	0	0
<i>sign</i>		<i>e*</i>				<i>m</i>									

The sign bit is **1**, so it's a negative number.

The biased exponent is **6**, so the true exponent is **$6 - 7 = -1$** .



Real Numbers – Floating point

Let's do some examples with our hypothetical 16-bit (4-bit exponent) IEEE 754 representation, with an exponent bias of $b = 7$.

Which number is represented by the following?

–	8	4	2	1	1024	512	256	128	64	32	16	8	4	2	1
1	0	1	1	0	0	1	1	0	0	1	1	0	0	0	0
<i>sign</i>					<i>e*</i>										
					<i>m</i>										

The sign bit is **1**, so it's a negative number.

The biased exponent is **6**, so the true exponent is **$6 - 7 = -1$** .

The mantissa with the hidden bit is



Real Numbers – Floating point

Let's do some examples with our hypothetical 16-bit (4-bit exponent) IEEE 754 representation, with an exponent bias of $b = 7$.

Which number is represented by the following?

–	8	4	2	1	1024	512	256	128	64	32	16	8	4	2	1
1	0	1	1	0	0	1	1	0	0	1	1	0	0	0	0
<i>sign</i>					<i>e*</i>										
					<i>m</i>										

The sign bit is **1**, so it's a negative number.

The biased exponent is **6**, so the true exponent is **$6 - 7 = -1$** .

The mantissa with the hidden bit is **$(1.01100110000)_2$** .



Real Numbers – Floating point

Let's do some examples with our hypothetical 16-bit (4-bit exponent) IEEE 754 representation, with an exponent bias of $b = 7$.

Which number is represented by the following?

2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}	2^{-7}	2^{-8}	2^{-9}	2^{-10}	2^{-11}	2^{-12}	2^{-13}	2^{-14}	2^{-15}

The sign bit is **1**, so it's a negative number.

The biased exponent is **6**, so the true exponent is **$6 - 7 = -1$** .

The mantissa with the hidden bit is **$(1.01100110000)_2$** .

The answer in fixed point is **$-(1.01100110000)_2 * 2^{-1}$**



Real Numbers – Floating point

Let's do some examples with our hypothetical 16-bit (4-bit exponent) IEEE 754 representation, with an exponent bias of $b = 7$.

Which number is represented by the following?

2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}	2^{-7}	2^{-8}	2^{-9}	2^{-10}	2^{-11}	2^{-12}	2^{-13}	2^{-14}	2^{-15}
0	1	0	1	1	0	0	1	1	0	0	0	0	0	0	0

The sign bit is **1**, so it's a negative number.

The biased exponent is **6**, so the true exponent is **$6 - 7 = -1$** .

The mantissa with the hidden bit is **$(1.01100110000)_2$** .

The answer in fixed point is **$-(1.01100110000)_2 * 2^{-1} = -(0.10110011)_2$** .



Real Numbers – Floating point

Let's do some examples with our hypothetical 16-bit (4-bit exponent) IEEE 754 representation, with an exponent bias of $b = 7$.

Which number is represented by the following?

2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}	2^{-7}	2^{-8}	2^{-9}	2^{-10}	2^{-11}	2^{-12}	2^{-13}	2^{-14}	2^{-15}
0	1	0	1	1	0	0	1	1	0	0	0	0	0	0	0

The sign bit is **1**, so it's a negative number.

The biased exponent is **6**, so the true exponent is **$6 - 7 = -1$** .

The mantissa with the hidden bit is **$(1.0110011)_2$** .

The answer in fixed point is **$-(1.0110011)_2 * 2^{-1} = -(0.10110011)_2$** .

The answer in denary is **$-(2^{-1} + 2^{-3} + 2^{-4} + 2^{-7} + 2^{-8})$**



Real Numbers – Floating point

Let's do some examples with our hypothetical 16-bit (4-bit exponent) IEEE 754 representation, with an exponent bias of $b = 7$.

Which number is represented by the following?

2^0	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}	2^{-7}	2^{-8}	2^{-9}	2^{-10}	2^{-11}	2^{-12}	2^{-13}	2^{-14}	2^{-15}
0	1	0	1	1	0	0	1	1	0	0	0	0	0	0	0

The sign bit is **1**, so it's a negative number.

The biased exponent is **6**, so the true exponent is **$6 - 7 = -1$** .

The mantissa with the hidden bit is **$(1.0110011)_2$** .

The answer in fixed point is **$-(1.0110011)_2 * 2^{-1} = -(0.10110011)_2$** .

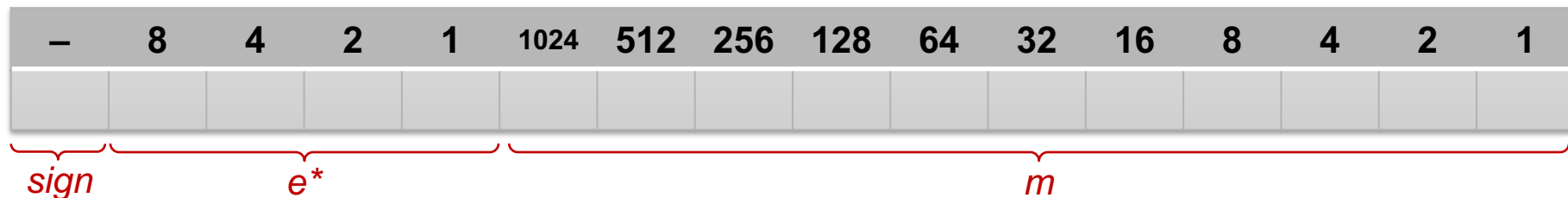
The answer in denary is **$-(2^{-1} + 2^{-3} + 2^{-4} + 2^{-7} + 2^{-8}) = -0.69921875$** .



Real Numbers – Floating point

Let's do some examples with our hypothetical 16-bit (4-bit exponent) IEEE 754 representation, with an exponent bias of $b = 7$.

How do we represent 142.5?

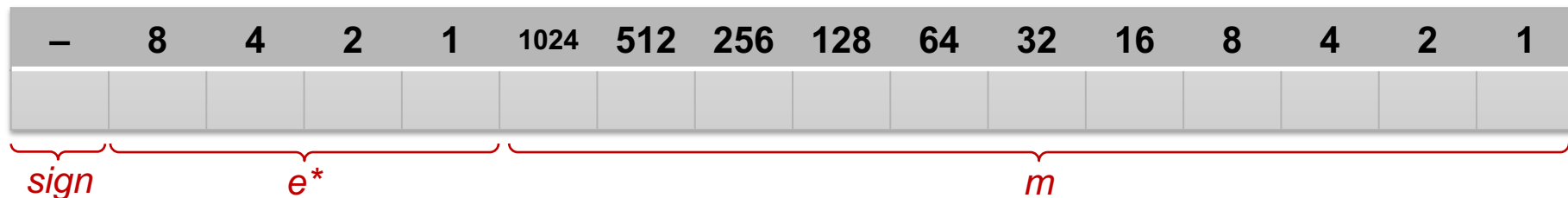




Real Numbers – Floating point

Let's do some examples with our hypothetical 16-bit (4-bit exponent) IEEE 754 representation, with an exponent bias of $b = 7$.

How do we represent 142.5?



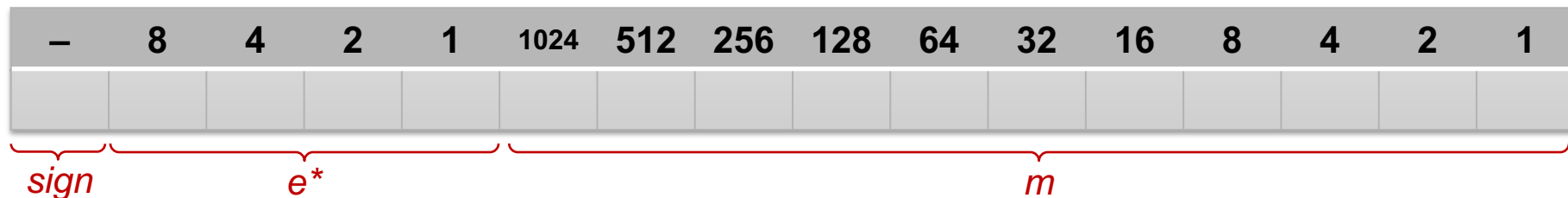
Convert it to fixed point:



Real Numbers – Floating point

Let's do some examples with our hypothetical 16-bit (4-bit exponent) IEEE 754 representation, with an exponent bias of $b = 7$.

How do we represent 142.5?



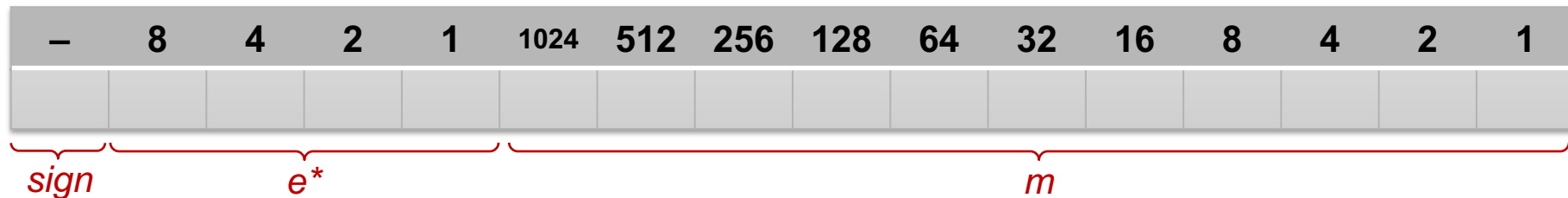
Convert it to fixed point: **10001110.1**.



Real Numbers – Floating point

Let's do some examples with our hypothetical 16-bit (4-bit exponent) IEEE 754 representation, with an exponent bias of $b = 7$.

How do we represent 142.5?



Convert it to fixed point: **10001110.1**.

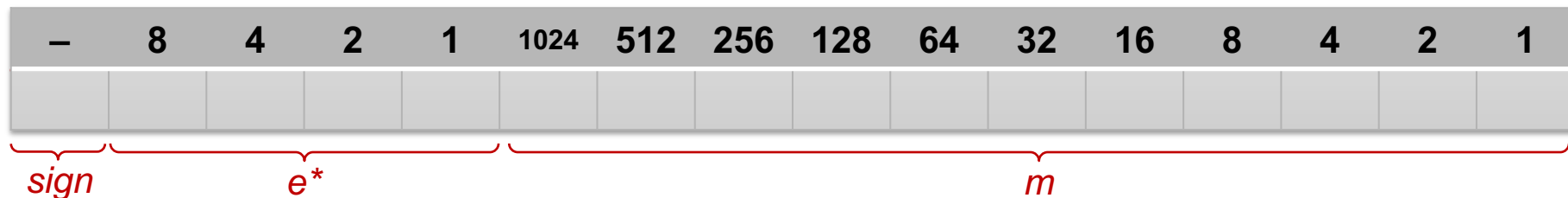
Write it in normalised scientific notation:



Real Numbers – Floating point

Let's do some examples with our hypothetical 16-bit (4-bit exponent) IEEE 754 representation, with an exponent bias of $b = 7$.

How do we represent 142.5?



Convert it to fixed point: **10001110.1**.

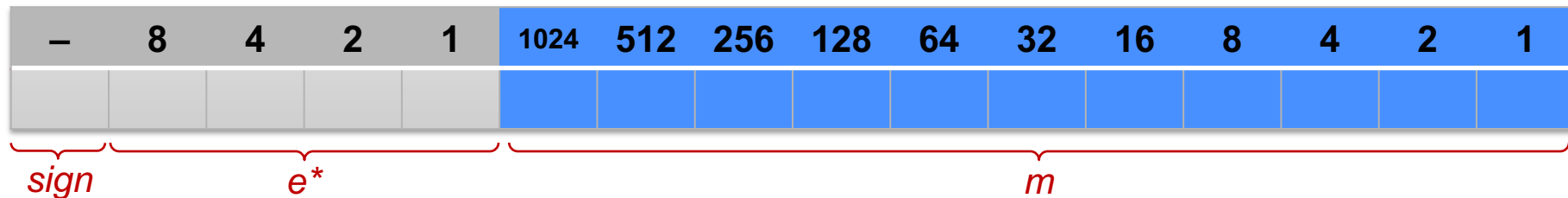
Write it in normalised scientific notation: **$(1.00011101)_2 * 2^7$** .



Real Numbers – Floating point

Let's do some examples with our hypothetical 16-bit (4-bit exponent) IEEE 754 representation, with an exponent bias of $b = 7$.

How do we represent 142.5?



Convert it to fixed point: **10001110.1**.

Write it in normalised scientific notation: **$(1.00011101)_2 * 2^7$** .

The mantissa is therefore



Real Numbers – Floating point

Let's do some examples with our hypothetical 16-bit (4-bit exponent) IEEE 754 representation, with an exponent bias of $b = 7$.

How do we represent 142.5?

–	8	4	2	1	1024	512	256	128	64	32	16	8	4	2	1
					0	0	0	1	1	1	0	1	0	0	0
<i>sign</i>		<i>e*</i>				<i>m</i>									

Convert it to fixed point: **10001110.1**.

Write it in normalised scientific notation: **$(1.00011101)_2 * 2^7$** .

The mantissa is therefore **00011101**.



Real Numbers – Floating point

Let's do some examples with our hypothetical 16-bit (4-bit exponent) IEEE 754 representation, with an exponent bias of $b = 7$.

How do we represent 142.5?

–	8	4	2	1	1024	512	256	128	64	32	16	8	4	2	1
					0	0	0	1	1	1	0	1	0	0	0
<i>sign</i>		<i>e*</i>				<i>m</i>									

Convert it to fixed point: **10001110.1**.

Write it in normalised scientific notation: **$(1.00011101)_2 * 2^7$** .

The mantissa is therefore **00011101**.

The true exponent is 7, so the biased exponent is



Real Numbers – Floating point

Let's do some examples with our hypothetical 16-bit (4-bit exponent) IEEE 754 representation, with an exponent bias of $b = 7$.

How do we represent 142.5?

–	8	4	2	1	1024	512	256	128	64	32	16	8	4	2	1
	1	1	1	0	0	0	0	1	1	1	0	1	0	0	0
<i>sign</i>		<i>e*</i>				<i>m</i>									

Convert it to fixed point: **10001110.1**.

Write it in normalised scientific notation: **$(1.00011101)_2 * 2^7$** .

The mantissa is therefore **00011101**.

The true exponent is 7, so the biased exponent is **$7 + 7 = 14$** .



Real Numbers – Floating point

Let's do some examples with our hypothetical 16-bit (4-bit exponent) IEEE 754 representation, with an exponent bias of $b = 7$.

How do we represent 142.5?

–	8	4	2	1	1024	512	256	128	64	32	16	8	4	2	1
	1	1	1	0	0	0	0	1	1	1	0	1	0	0	0
<i>sign</i>	<i>e*</i>				<i>m</i>										

Convert it to fixed point: **10001110.1**.

Write it in normalised scientific notation: **$(1.00011101)_2 * 2^7$** .

The mantissa is therefore **00011101**.

The true exponent is 7, so the biased exponent is **$7 + 7 = 14$** .

The number is positive, so the sign bit is **0**.



Real Numbers – Floating point

Let's do some examples with our hypothetical 16-bit (4-bit exponent) IEEE 754 representation, with an exponent bias of $b = 7$.

How do we represent 142.5?

–	8	4	2	1	1024	512	256	128	64	32	16	8	4	2	1
0	1	1	1	0	0	0	0	1	1	1	0	1	0	0	0
<i>sign</i>		<i>e*</i>				<i>m</i>									

Convert it to fixed point: **10001110.1**.

Write it in normalised scientific notation: **$(1.00011101)_2 * 2^7$** .

The mantissa is therefore **00011101**.

The true exponent is 7, so the biased exponent is **$7 + 7 = 14$** .

The number is positive, so the sign bit is **0**.



Real Numbers

Floating point is a gold standard for representing real numbers in binary on a computer.

It allows us to represent a wide array of numbers dynamically, as we are able to shift the binary point left or right to get the precision we require.

However, some numbers – no matter how much precision we have – can never be represented in binary.

1	0.5	0.25	0.125	0.0625
0	0	0	0	0

0.5	0.25	0.125	0.0625	0.03125
0	0	0	0	1



Real Numbers

Even with a huge number of bits, we will not be able to represent many real numbers.

In denary, numbers such as π and $1/3$ cannot be represented without an infinite number of digits!

In binary, numbers such as those above, and others such as 0.1 , cannot be represented without an infinite number of bits!

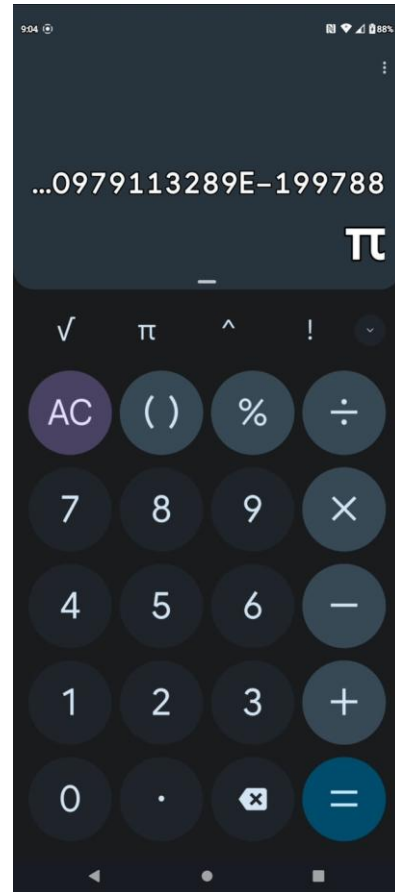
2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}	2^{-7}	2^{-8}	2^{-9}	2^{-10}	2^{-11}	2^{-12}	2^{-13}	2^{-14}	2^{-15}	...
0	0	0	1	1	0	0	1	1	0	0	1	1	0	0	...



Real Numbers

Research is ongoing in the field of represent *exact real numbers*. This includes the development of data types in programming languages that allow one to represent *any* real number, not just those that can be represented using floating points.

(In fact I wrote my thesis on this 😊)





Topic Recap



- 01 Everything on a computer is represented in **binary**.
- 02 Binary is the **base 2** number system that consists only of 0s and 1s.
- 03 Colour and text can be represented in different ways, depending on **how many bits** we have available.
- 04 We can perform **arithmetic** (+,<<,>>,-) operations on binary words.
- 05 Negative numbers can be represented either by using a **sign bit** or by using the **two's complement** representation.
- 06 Real numbers can be represented either by using **fixed point** or **floating** representation.



Further Reading

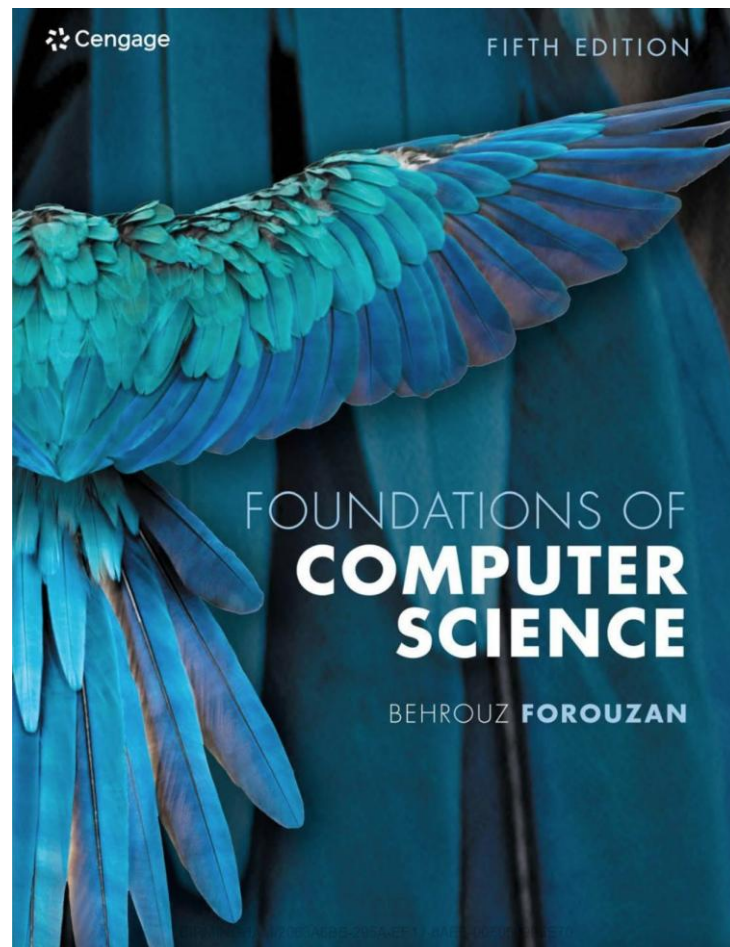
Forouzan (2023) *Foundations of Computer Science*. Cengage Learning. [[link](#)]

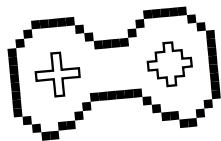
Chapter 1: Introduction

Chapter 2: Number Systems

Chapter 3: Data Storage

Chapter 4: Operations on Data





Thanks!

Do you have any questions?

t.waughambridge@bham.ac.uk

Office Hours:

13:00-16:30 Mondays in CS/109



CREDITS: This presentation template was created by
Slidesgo, and includes icons by **Flaticon**, and infographics
& images by **Freepik**